

The sequence-dependent job sequencing and tool switching problem with non-identical parallel machines

Achmad Pratama Rifai^{a1}, Wangi Pandan Sari^{a2}

^a Universitas Gadjah Mada, Indonesia.

^{a1} achmad.p.rifai@ugm.ac.id; ^{a2} wangipandansari@ugm.ac.id

Abstract:

This study explores the job sequencing and tool switching problem with sequence-dependent setup times on non-identical parallel machines (SDSSP-NPM). The problem involves diverse parallel machines with varying tool magazine capacities, processing times, and sequence-dependent setup times. This extension is particularly relevant in manufacturing, where production lines consist of machines capable of handling different job requirements. The model addresses three objectives: minimizing total setup time, total flow time, and makespan. To tackle the SDSSP-NPM, a two-stage metaheuristic approach is introduced, combining adaptive large neighborhood search (ALNS) and simulated annealing (SA). ALNS is employed to solve the job sequencing subproblem (SP), while SA, coupled with the keep tool needed soonest (KTNS) algorithm, handles the tool switching subproblem (TP). The proposed method is rigorously tested across 640 instances with varying complexities. Additionally, sensitivity analysis is conducted to assess the impact of two key parameters: the number of required tools for each job and the duration of switching time on the method's performance. The experimental results demonstrate the effectiveness of the proposed method, outperforming other recent approaches by generating solutions with superior objectives.

Key words:

Job sequencing and tool switching, non-identical parallel machines, sequence-dependent setup time, adaptive large neighborhood search, simulated annealing.

1. Introduction

Efficient job scheduling in manufacturing and industrial operations is essential for optimizing production processes, reducing costs, and enhancing overall productivity. To meet these critical objectives, various manufacturing systems and strategies have been developed. Among these, Flexible Manufacturing Systems (FMS) are commonly utilized as a strategic response to these production challenges (Grassi et al., 2023). The adoption of FMS has become imperative for addressing several

pressing needs, including managing a broader spectrum of product variations, accommodating shifting customer demands, and navigating the complexities of dynamic and fluctuating production requirements (Javaid et al., 2022).

FMS represents a highly integrated assembly of numerically controlled machines, interlinked through an automated material handling system (Javaid et al., 2022). One of the defining features of FMS is the integration of numerically controlled machines, which play a pivotal role in executing diverse

To cite this article: Rifai, A. P., & Sari, W. P. (2026). The sequence-dependent job sequencing and tool switching problem with non-identical parallel machines. *International Journal of Production Management and Engineering*, 14(2), e23073. <https://doi.org/10.4995/ijpme.2026.23073>

manufacturing tasks. These machines are equipped with a tool magazine that serves as a repository for various tools, each with specific functions and capabilities. As product variations increase to a level where the quantity of tools required to complete a job sequence exceeds the magazine capacity, the need for tool changes becomes inevitable. Tool changes are inherently influenced by the specific tools required for upcoming tasks, thus making them intricately linked to the sequencing of jobs. Hence, it is commonly referred to as the Job Sequencing and Tool Switching Problem (SSP).

SSP is a well-known combinatorial optimization problem that extends across various industries. In aerospace sector, the problem finds relevance in the production of various components, including vehicle body parts, landing gears, aircraft components, and internal parts (Shirazi & Frizelle, 2001). In the electronics industry, SSP comes into play when arranging the sequencing of printed circuit boards (PCBs) (Ghrayeb et al., 2003; Raduly-Baka and Nevalainen, 2015). In computer science, the SSP exhibits similarities to k-server problems, a concept employed in processing bulk requests (Privault and Finke, 2000). In packaging industry, SSP is applicable in optimizing pharmaceutical packaging, such as efficiently arranging pills into boxes (Mütze, 2014). It is also leveraged for enhancing processes in multi-color printing as discussed in (Burger et al., 2015) and (Iori et al., 2022). SSP can be also implemented in other areas such as order picking systems with cranes (Schwerdfeger and Boysen, 2017) and mailroom inserting systems planning (Adjashvili et al., 2015; Furrer and Mütze, 2017).

Although extensive research has been conducted on SSP, the majority of previous studies predominantly centered on the uniform SSP constructed with the assumption of uniform and sequence-dependent set-up time. In practical manufacturing scenarios, it can be crucial to consider non-uniform setup times where the removal and installation of tools might be varied depending on the specific tools involved in the operations. In addition, the setup time required to switch from one tool to another is not constant; it depends on the specific sequence of the operations, i.e., tool removal, tool positioning, tool insertion, cleaning up, and inspection of tool mounting, as well as influenced by the type of tools and machines. As highlighted in Calmels (2019), most existing research in SSP assumes uniform switching times and neglects the practical scenarios where uniform set-up times are implausible. The complexity of the problem is

heightened by the presence of non-identical parallel machines in manufacturing settings, which exhibit distinct processing capabilities, tool capacities, and processing times.

This study addresses a sequence-dependent job sequencing and tool switching problem (SDSSP) with non-identical parallel machines which is particularly well-suited for dealing with the challenges found in modern production settings. The main contribution of the study lies in the two-stage heuristic procedure for a non-uniform and sequence-dependent SSP with non-identical parallel machine.

The SSP is defined by two distinct subproblems: 1) tool switching problem (TP) and 2) job sequencing problem (Rifai et al., 2022a). Tailored methodologies are developed for each subproblem. In the TP subproblem, a combined approach involving KTNS and simulated annealing (KTNS + SA) is proposed to determine the precise tool switching schedule, identifying near-optimal tool pairs. For job sequencing, an adaptive large neighborhood search (ALNS) algorithm is used to find near-optimal job sequences while minimizing setup times, total flow time, and makespan. The ALNS focuses on enhancing neighborhood search through various heuristics and includes additional heuristics tailored to reduce setup times.

Finally, this paper is structured as follows. Section 2 discusses the review of relevant literatures. Section 3 presents the flow formulation of the sequence-dependent job sequencing and tool switching problem with non-identical parallel machines. Section 4 presents the proposed metaheuristics, while Section 5 provides the experimental results in detail. Finally, Section 6 provides the conclusions of this study as well as several suggestions for future research.

2. Literature review

Numerous formulations of the job sequencing and tool switching problem and its various iterations have been developed over time. The most recognized among them is the uniform SSP, firstly introduced in (Tang and Denardo, 1988). In this problem, it is assumed that setup times for tool changes remain uniform and constant, regardless of the specific jobs in the production schedule. Tang and Denardo introduced a Mixed Integer Linear Programming (MILP) formulation for the uniform SSP. They

proposed greedy perturbation method, applying the ‘Keep Tool Needed Soonest’ (KTNS) to find solutions for the job sequence problem. [Catanzaro et al. \(2015\)](#) introduced improved ILP formulations with tighter linear programming (LP) relaxation for the SSP. They explored a combinatorial interpretation and strengthened valid inequalities to further enhance the lower bounds provided by the LP relaxation. While exact methods are preferred to find the optimal solution for SSP and technology advancements have enabled their use for larger instances, heuristic techniques are often favored for their efficient delivery of near-optimal solutions. [Chaves et al. \(2016\)](#) introduced a Clustering Search (CS) approach that combines a Biased Random Key Genetic Algorithm (BRKGA) with Variable Neighborhood Search to improve solution quality in the basic SSP. [Paiva and Carvalho \(2017\)](#) presented an Iterated Local Search (ILS) metaheuristic to solve SSP in a single machine. Their approach incorporated a novel graph representation and block grouping which showed to outperformed other methods included the one proposed by ([Chaves et al., 2016](#)) in a different study. [Atta et al. \(2019\)](#) introduced enhancements to the Harmony Search algorithm specifically aimed at addressing the tool indexing problem. They did not address the complexities of the job sequencing and switching problem but instead concentrated on optimizing the allocation management of tools within the magazine’s pockets to minimize job processing time on the machines.

While uniform SSP predominantly addresses a single machine, multiple machines SSP extend the problem by considering uniform or non-uniform magazine capacities. Uniform magazine capacities mean that each machine has an identical tool magazine with the same tool-holding capacity. In a study by [Khan et al. \(2000\)](#), a practical greedy heuristic was designed for solving the SSP when dealing with two identical parallel machines. [Özpeynirci et al. \(2016\)](#) explored the SSP problems to solve for unrelated parallel machines with a primary goal of minimizing the makespan. They found the need for heuristic methods due to the challenges posed by mathematical models and developed a tabu search (TS) algorithm to efficiently identify near-optimal solutions in reduced time. [Beezão et al. \(2017\)](#) explored the scheduling of identical parallel machines with tooling constraint (IPMTC). They proposed two Integer Linear Programming (ILP) models, derived from formulations by [Tang and Denardo \(1988\)](#) and [Laporte et al. \(2004\)](#). [Dang et al. \(2021\)](#) expanded the model to incorporate multiple

criteria, including tardiness and tool set-up time in their MILP framework. They also introduced a new metaheuristic approach that combined a genetic algorithm (GA) and integer linear program (ILP), demonstrating a superior performance compared to the MILP model. While [Beezão et al. \(2017\)](#) aimed to minimize the total makespan, the study by [Dang et al. \(2021\)](#) focused on reducing the total tardiness and tool setup time. In a recent investigation, these problems were extended to incorporate unsupervised production, where production can continue without supervision, but tool switches were constrained to occur solely during supervised hours ([Dang et al., 2023](#)). To solve the problems, MILP model and GA were introduced. While the MILP model was effective for small-scale instances (up to twenty-five operations), the proposed GA showed high-quality solutions for larger industry-sized instances, all within a reasonable time.

The presence of non-identical parallel machines in manufacturing settings increases the complexity of the problem. This implies the machines may have different magazine capacities and different tools so the processing time of a job may not be uniform for all machines. This variation, known as SSP-NPM, has been relatively underexplored in the literature. [Van Hop and Nagarur \(2004\)](#) consider the scheduling of printed circuit packs on non-identical parallel sequencers that are equipped with a number of dispensing heads that have to be loaded with the required input tapes. To solve the problem, they employed a CGA with the objective of minimizing the total makespan. In a recent study, [Calmels \(2022\)](#) introduced non-identical parallel machines with different tool magazine capacity and setup times. The study aimed to balance three competing objectives: reducing the overall job flowtime, minimizing the makespan, and decreasing the total tool changes. To accomplish this, they utilized a Mixed Integer Linear Programming (MILP) model. However, the approach could only optimally address small-size problem instances. To address larger large problems, iterated local search (ILS) methods were proposed. Computation experiments demonstrated the ability of the methods to generate good solutions within reasonable computational timeframe. Different approach to SSP-NPM was proposed by [Cura \(2023\)](#). The study introduced a hybrid method combining GA and a local search (LS) method and presented two variations: 1) “Randomly Remove Tool” (RRT) and 2) “Keep Tool Needed Soonest” (KTNS). Both versions demonstrated the ability to yield provide good solutions within reasonable time.

Notably, KTNS greatly enhanced solution quality, especially for larger problem instances, while RRT demonstrated superior computational efficiency.

The scenario of sequence-dependent setup time is seldom mentioned in the context of SSP, as observed by Calmels (2019). Only a handful of articles have explicitly stressed the importance of considering the non-uniform SSP. Privault and Finke (1995) were pioneers in exploring sequence-dependent setup times within the SSP model, although they did not propose a mathematical model. In recent years, Mara et al. (2023) provided the mathematical model which exclusively considers the inclusion of sequence dependent setup time with five-index formulation model. Rifai et al. (2022a) proposed a two-stage heuristic based on ALNS and SA for the SDSSP in a single machine setting. The study was continued by Rifai et al. (2022b) which developed multi-objective model of SDSSP in a single machine to minimize both total setup time and tardiness in which Multi-objective Adaptive Large Neighborhood Search (MOALNS) was proposed to solve the job sequencing subproblem.

Albeit several studies have extensively explored the SSP and SDSSP in a single machine environment, there are only a few studies concerned on the problem with parallel machines, such as the studies by Özpeynirci et al. (2016), Beezão et al. (2017), and Gökğür et al. (2018). Moreover, only a handful of studies which specifically focus on the SSP in non-identical parallel machines (SSP-NPM). Calmels (2022) incorporated machine-dependent processing and tool switching times into the SSP involving non-identical parallel machines. However, it is important to note that the non-uniform setup time primarily depends on the machine used for job processing rather than the previously installed tools. In our previous study, we addressed this gap by introducing an optimization model that can accommodate the sequence-dependent setup time in a single machine (Mara et al., 2023; Rifai et al., 2022a).

In this study, we extend the issue of sequence-dependent setup time and consider its implementation in non-identical parallel machines. The SDSSP in non-identical parallel machines notably has more complexity due to the inclusion of the decision of optimal machine assignments, in addition to the determination of job sequencing and tool switching schedules. The contributions of this work are two folds: (a) formulating a mixed integer linear programming (MILP) model for the SDSSP-NPM,

and (b) developing a two-stage metaheuristics based on ALNS and SA for solving the job sequencing and tool switching subproblems in SDSSP-NPM, respectively.

3. Problem formulation

This section provides a description of the SDSSP with non-identical parallel machines. The mathematical formulation for the problem is presented firstly in Section 3.1, including the introduction of three objectives (minimizing total setup time, total flow time, and makespan). Then, in Section 3.2, we present a case illustration of the SDSSP with non-identical parallel machines and trade-off relationships between the three objectives.

3.1. Mathematical model

The SDSSP with non-identical parallel machines concerns on the allocation jobs $\mathcal{J}$ to a set of machines $\mathcal{M}$, the sequencing process of jobs $\mathcal{J}$ in each machine, and the installation of tools $\mathcal{T}$ to process the assigned jobs. A pre-defined subset of tools $\mathcal{T}_j$ is required to process job j in any machine $m \in \mathcal{M}$. There are several available machines $m \in \mathcal{M}$ which are non-identical, in which the processing time p_j^m to process a job in machine m can be different with the processing time to process the same job in different machine.

Each machine $m \in \mathcal{M}$ is equipped with a tool magazine consisting of a set of tool slots $\mathcal{V}_m$ with finite magazine capacity C^m , in which any tool $t \in \mathcal{T}$ can be loaded in any slot $v \in \mathcal{V}_m$. In practical manufacturing environments, the magazine capacity is typically much smaller than the total number of available tools ($C^m < T$), which restricts the number of tools that can be installed simultaneously. Consequently, tool switching operations become unavoidable when processing jobs requiring different tool sets.

In this study, the tool magazine C^m is also limited to an assumption that $\max_{j \in \mathcal{J}^m} \{|\mathcal{T}_j|\} \leq C^m$. This assumption specifies that the magazine capacity of a machine C^m should be larger than the number of tools required to process the job that can be accommodated by that machine $j \in \mathcal{J}^m$. This assumption reflects realistic industrial conditions in which a single job can always be processed without interruption once it has started. As a result, tool switching is not allowed during job processing, and

all tool replacement operations are performed only after the completion of the previous job and before the start of the subsequent job.

Under this setting, a non-uniform setup time τ_{tu}^m for replacing tool $t \in \mathcal{T}$ with tool $u \in \mathcal{T} \setminus \{t\}$ in tool slot v is required during tool switching, in which the value is sequence dependent according to the replaced tool $t \in \mathcal{T}$ and to be installed tool $u \in \mathcal{T}_j$. Furthermore, setup times are machine dependent, reflecting differences in tool changers, mechanical configurations, or automation levels among machines. Hence, the value of τ_{tu}^m may vary both across tool pairs and across machines. This sequence-dependent and machine-dependent nature of setup times introduces strong interdependencies between job sequencing, machine allocation, and tool switching decisions. Two identical job sequences assigned to different machines may result in significantly different setup times and completion times, while alternative sequences with more frequent tool changes may still yield lower total setup time if favorable tool replacement pairs are selected.

Additionally, not all machines are capable of processing all jobs. Certain machines may be technologically incompatible with specific jobs, while other jobs can be processed on multiple machines but with different processing times p_j^m . This issue can be solved by simply setting the value of setup time of tools in machine that cannot be processed job j with very large number $\tau_{tu}^k = G$. $k \in \mathcal{M}$ is the machine that cannot be used to process job j , $u \in \mathcal{T}_j$ is tool required to process job j , and G is a very large number. In this case, assigning job j to machine k will be avoided since it will result on very high value of the objectives total setup time σ , total flow time TFT, and makespan Fmax. Consequently, such infeasible allocations are naturally avoided by the optimization process.

This penalty-based modeling strategy offers a computationally efficient alternative to introducing additional binary decision variables and compatibility constraints, which would otherwise substantially increase model size and solution time. By embedding infeasibility directly into the setup-time structure, the model preserves formulation compactness while maintaining correct solution behavior.

The complete set of indices, parameters, and decision variables used in the mathematical formulation of the SDSSP with non-identical parallel machines is summarized in Table 1.

Table 1. Notations of the mathematical model.

Sets and indices	
$\mathcal{J}$	Set of jobs, $\mathcal{J} = \{1, 2, 3, \dots, J\}$
$\mathcal{M}$	Set of machines, $\mathcal{M} = \{1, 2, 3, \dots, M\}$
$\mathcal{T}$	Set of tools, $\mathcal{T} = \{1, 2, 3, \dots, T\}$
$\mathcal{V}_m$	Set of tools slots in machine m , $\mathcal{V}_m = \{1, 2, 3, \dots, C^m\}$
$\mathcal{T}_j$	Set of tools required to process job j , $\mathcal{T}_j \in \mathcal{T}$
i, j	Index of job, $i, j \in \mathcal{J}$
m	Index of machine, $m \in \mathcal{M}$
t, u	Index of tool, $t, u \in \mathcal{T}$
v	Index of tool slot, $v \in \mathcal{V}_m$
Parameters	
J	Number of jobs
M	Number of machines
T	Number of available tools
C^m	Tool magazine capacity of machine m
p_j^m	Processing time of job j in machine m
τ_{tu}^m	Sequence dependent setup time required for replacing tool $t \in \mathcal{T}$ with tool $u \in \mathcal{T} \setminus \{t\}$ in machine m
G	Very large number/Variables
x_{ij}^m	Binary decision variable, 1 if job j follows job i immediately in machine m , 0 otherwise
$y_{jm}^{t,v}$	Binary decision variable, 1 if tool t is inserted into slot v when job j is processed in machine m , 0 otherwise
$z_{ijm}^{t,u,v}$	Binary decision variable, 1 if tool u is inserted at slot v to process job j in machine m by replacing tool t which is used to process the predecessor job i , 0 otherwise
f_j^m	Completion time of job j in machine m
g_j	Additional variable for subtour elimination
w_j^m	Additional binary variable for makespan linearization, 1 if job j processed in machine m has the highest completion time f_j^m , 0 otherwise
B_j^m	Additional variable for ensuring that each job is assigned to exactly one machine, 1 if job j is processed in machine m , 0 otherwise
σ	Total setup time
s	Total number of switches
TFT	Total flow time
$Fmax$	Makespan

Here, we also formulate set $\mathcal{H} = \mathcal{J} \cup \{j_0\}$ and $\mathcal{R} = \mathcal{T} \cup \{\mathcal{T}_0\}$, where $\{0\}$ is a dummy job introduced to represent the starting and stopping points of the machining operations. This model is an extension of the five-index formulation model for SDSSP with a single machine described in [Mara et al. \(2023\)](#). As such, several assumptions held in the SDSSP are also valid in the SDSSP with non-identical machines, such as (i) each tool can be installed into any magazine slot, (ii) the workpiece setup time is included in the job processing time, (iii) uniform tool size, (iv) a single tool only occupies a single magazine slot, (v) there is only a single copy of each tool, and (vi) at the beginning, the magazines are empty, thus it requires the tool to be inserted with initial setup time τ_{0u}^m where $u \in \mathcal{T}_j$, $m \in \mathcal{M}$.

The objectives of the model are adopted from the study of [Calmels \(2022\)](#) which are to minimize the total setup time σ , total flow time TFT , and makespan F_{max} . In this study, the problem is formulated and solved as a single-objective optimization model, where each objective is evaluated independently rather than through a multi-objective approach. Consequently, three separate optimization models are developed for the sequence-dependent scheduling problem on non-identical parallel machines. The complete mathematical formulation is provided below.

Objective function

$$\min \sigma = \sum_{m \in \mathcal{M}} \sum_{i \in \mathcal{H} \setminus \{j\}} \sum_{j \in \mathcal{J} \setminus \{i\}} \sum_{t \in \mathcal{R}} \sum_{u \in \mathcal{T}} \sum_{v \in \mathcal{V}_m} \tau_{tu}^m z_{ijm}^{t,u,v} \quad (1)$$

$$\min TFT = \sum_{m \in \mathcal{M}} \sum_{j \in \mathcal{J}} f_j^m \quad (2)$$

$$\min F_{max} = \sum_{m \in \mathcal{M}} \sum_{j \in \mathcal{J}} w_j^m f_j^m \quad (3)$$

Subject to:

$$\sum_{m \in \mathcal{M}} \sum_{j \in \mathcal{H} \setminus \{i\}} x_{ij}^m = 1 \quad \forall i \in \mathcal{J} \quad (4)$$

$$\sum_{j \in \mathcal{H}} x_{ij}^m = \sum_{j \in \mathcal{H}} x_{ji}^m \quad \forall m \in \mathcal{M}, i \in \mathcal{J} \quad (5)$$

$$\sum_{j \in \mathcal{J}} x_{0j}^m = 1 \quad \forall m \in \mathcal{M} \quad (6)$$

$$g_i - g_j + J(x_{ij}^m) \leq J - 1 \quad \forall m \in \mathcal{M}, i \in \mathcal{J}, j \in \mathcal{J}, i \neq j \quad (7)$$

$$\sum_{t \in \mathcal{T}} \sum_{v \in \mathcal{V}_m} y_{jm}^{t,v} \leq C^m \quad \forall m \in \mathcal{M}, j \in \mathcal{J} \quad (8)$$

$$\sum_{v \in \mathcal{V}_m} y_{jm}^{t,v} = \sum_{i \in \mathcal{H} \setminus \{j\}} x_{ij}^m \quad \forall m \in \mathcal{M}, j \in \mathcal{J}, t \in \mathcal{T}_j \quad (9)$$

$$\sum_{v \in \mathcal{V}_m} y_{jm}^{t,v} \leq \sum_{i \in \mathcal{H} \setminus \{j\}} x_{ij}^m \quad \forall m \in \mathcal{M}, j \in \mathcal{J}, t \notin \mathcal{T}_j \quad (10)$$

$$\sum_{t \in \mathcal{T}} y_{jm}^{t,v} = \sum_{i \in \mathcal{H} \setminus \{j\}} x_{ij}^m \quad \forall m \in \mathcal{M}, j \in \mathcal{J}, v \in \mathcal{V}_m \quad (11)$$

$$x_{ij}^m + y_{jm}^{u,v} - y_{im}^{u,v} \leq 1 + \sum_{t \in \mathcal{T} \setminus \{u\}} z_{ijm}^{t,u,v} \quad \forall m \in \mathcal{M}, i \in \mathcal{J}, j \in \mathcal{J}, i \neq j, u \in \mathcal{T}, v \in \mathcal{V}_m \quad (12)$$

$$x_{0j}^m + y_{jm}^{u,v} \leq 1 + z_{0jm}^{0,u,v} \quad \forall m \in \mathcal{M}, j \in \mathcal{J}, u \in \mathcal{T}, v \in \mathcal{V}_m \quad (13)$$

$$x_{ij}^m \geq z_{ijm}^{t,u,v} \quad \forall m \in \mathcal{M}, i \in \mathcal{H}, j \in \mathcal{J}, i \neq j, t \in \mathcal{T}, u \in \mathcal{T}, v \in \mathcal{V}_m \quad (14)$$

$$y_{im}^{t,v} \geq z_{ijm}^{t,u,v} \quad \forall m \in \mathcal{M}, i \in \mathcal{J}, j \in \mathcal{J}, i \neq j, t \in \mathcal{T}, u \in \mathcal{T}, v \in \mathcal{V}_m \quad (15)$$

$$\sum_{u \in \mathcal{T}_j} \sum_{v \in \mathcal{V}_m} z_{ijm}^{t,u,v} \leq 1 \quad \forall m \in \mathcal{M}, i \in \mathcal{J}, j \in \mathcal{J}, i \neq j, t \in \mathcal{T} \quad (16)$$

$$\sum_{t \in \mathcal{T}} \sum_{v \in \mathcal{V}_m} z_{ijm}^{t,u,v} \leq 1 \quad \forall m \in \mathcal{M}, i \in \mathcal{J}, j \in \mathcal{J}, i \neq j, u \in \mathcal{T}_j \quad (17)$$

$$\sum_{j \in \mathcal{J} \setminus \{i\}} \sum_{t \in \mathcal{R}} \sum_{u \in \mathcal{T}} \sum_{v \in \mathcal{V}_m} z_{ijm}^{t,u,v} \leq C^m \quad \forall m \in \mathcal{M}, i \in \mathcal{H} \quad (18)$$

$$f_j^m \geq x_{0j}^m (p_j^m + \sum_{t \in \mathcal{R}} \sum_{u \in \mathcal{T}} \sum_{v \in \mathcal{V}_m} \tau_{tu}^m z_{0jm}^{t,u,v}) \quad \forall m \in \mathcal{M}, j \in \mathcal{J} \quad (19)$$

$$f_j^m \geq x_{ij}^m (f_i^m + p_j^m + \sum_{t \in \mathcal{R}} \sum_{u \in \mathcal{T}} \sum_{v \in \mathcal{V}_m} \tau_{tu}^m z_{ijm}^{t,u,v}) \quad \forall m \in \mathcal{M}, i \in \mathcal{J}, j \in \mathcal{J}, i \neq j \quad (20)$$

$$\sum_{m \in \mathcal{M}} B_j^m = 1 \quad \forall j \in \mathcal{J} \quad (21)$$

$$f_j^m \leq G(B_j^m) \quad \forall m \in \mathcal{M}, j \in \mathcal{J} \quad (22)$$

$$F_{max} \geq f_j^m \quad \forall m \in \mathcal{M}, j \in \mathcal{J} \quad (23)$$

$$\sum_{j \in \mathcal{J}} \sum_{m \in \mathcal{M}} w_j^m = 1 \quad (24)$$

$$x_{ij}^m \in \{0, 1\} \quad \forall m \in \mathcal{M}, i \in \mathcal{H}, j \in \mathcal{H}, i \neq j \quad (25)$$

$$y_{jm}^{t,v} \in \{0, 1\} \quad \forall m \in \mathcal{M}, j \in \mathcal{J}, t \in \mathcal{T}, v \in \mathcal{V}_m \quad (26)$$

$$z_{ijm}^{t,u,v} \in \{0, 1\} \quad \forall m \in \mathcal{M}, i \in \mathcal{H}, j \in \mathcal{J}, i \neq j, t \in \mathcal{R}, u \in \mathcal{T}, v \in \mathcal{V}_m \quad (27)$$

$$w_j^m \in \{0, 1\} \quad \forall m \in \mathcal{M}, j \in \mathcal{J} \quad (28)$$

$$f_j^m \geq 0 \quad \forall m \in \mathcal{M}, j \in \mathcal{J} \quad (29)$$

Equations (1-3) mathematically presents the objective functions of this formulation, which are minimizing total setup time σ , total flow time TFT , and makespan $Fmax$. Equations (4-7) specify the job sequence akin to a Vehicle Routing Problem (VRP) sequence. Equations (4-5) maintains the flow of jobs and ensures consistency in entering and leaving a job. Equation (6) ensures that the processing in each machine always starts from dummy job $\{0\}$. Equation (7) is subtour elimination constraint which effectively prevents the formation of smaller cycles that do not include the dummy job, thereby ensuring a single continuous route.

Equations (8-18) are tool switching constraints. Equation (8) enforces the tool magazine's capacity for each machine C^m . Equations (9-11) guarantee each tool in a machine is used once per job and occupies only one magazine slot. Equations (12-18) ensure the job sequence's continuity and tool switch schedule within a machine. Equations (12-13) confirm all required tools for a job are available during its processing in which Equation (12) dictates mid-process installment while Equation (13) dictates initial installment. Equations (14-15) regulate that tool switches only happen in active job transitions. Equations (16-17) restrict a tool's replacement to one occurrence during each job transition (i,j) . Equation (18) limits the maximum number of tool switches in a job transition to the machine's magazine slots.

Equations (19-20) govern the completion time of each job f_j^m . Equation (19) specifically regulates the completion time of the first job in each machine, while Equation (20) is used for the rest of jobs. This equation limits that the completion time of job f_j^m must be greater than the summation of completion time of predecessor job in the same machine f_i^m , its processing time p_j^m , and the setup time of tools to process the job. Equations (21-22) are constraints to ensure that if the value of f_j^m for a particular job j and machine m is non-zero, then the values of $f_j^{m'}$ for the same job j but different machine $m' \neq m$ must be zero. Equations (23-24) are used for linearization of makespan calculation. Equations (25-28) define binary variables x_{ij}^m , $y_{jm}^{t,v}$, $z_{ijm}^{t,u,v}$, and w_j^m . Lastly, Equation (29) ensures the integrality of continuous decision variable f_j^m , essential for total flow time TFT and makespan $Fmax$ calculations.

3.2. Case illustration

Table 2 shows an example of instances of the SSP with non-identical machine (Calmels, 2022). p_j^1 is the processing time of job j in machine 1, while p_j^2 is the processing time in machine 2. Noted that $j=1, 2, 4$ could not be processed in machine 2 since the number of required tools to process these jobs exceed the tool magazine capacity of machine 2. The jobs are processed sequentially on each machine. For this illustration, it is assumed that the tools required to process the first tool are already installed in the magazine, with $\tau_{0u}^m = 0$. Hence, the calculation of initial setup time is omitted in this example for simplicity.

Table 2. An instance with $M=2, J=6, T=9, C^1=4, C^2=3$.

Jobs	1	2	3	4	5	6
p_j^1	1	3	5	4	6	1
p_j^2	-	4	-	-	3	1
Tools	1	1	2	1	3	1
	4	3	6	5	5	2
	8	5	7	7	8	4
	9		8	9		

One of the key properties of SSP is that the magazine capacity is not enough to contain all tools required to process all jobs. Hence, tool switching is required. A certain amount of time is required to unmount the previous tools and install the new tools to process the next job, called as the setup time τ . The setup time is sequence dependent, depending on the replaced and to be installed tools. In this illustration, it is assumed that the setup time is symmetric in which the setup time to remove tool 1 and install tool 2 is the same as the time to remove tool 2 and install tool 1 $\tau_{1,2}^1 = \tau_{2,1}^1$. However, the setup time of the same tools pair may be different between machines $\tau_{1,2}^1 \neq \tau_{1,2}^2$. The sequence-dependent setup times for the problem above are presented in Table 3.

Two optimal solutions are presented for the above problem, as depicted in Figure 1. The first solution (Figure 1a) is the optimal solution with minimizing the number of tool switches, hence ignoring the detail of tool switching schedule. Meanwhile, the second solution (Figure 1b) is obtained by minimizing the total flow time as the objective.

The first solution has total setup time $\sigma=19$, total flow time $TFT=74$, makespan $Fmax=26$, and number of switches $s=6$. Meanwhile, the second

solution has total setup time $\sigma=16$, total flow time $TFT=53$, makespan $F_{\max}=19$, and number of switches $s=9$.

The switching schedule in solution (a) is generated under the assumption of a uniform SSP using the Keep Tool Needed Soonest (KTNS) rule, whose primary objective is to minimize the number of tool switches. As a result, solution (a) indeed produces a smaller number of switches compared to solution (b). However, this strategy ignores the sequence-dependent nature of setup times, and therefore fails to identify optimal tool replacement pairs. Consequently, a lower number of switches does not necessarily translate into lower total setup time or better overall system performance.

On the contrary, the sequence-dependent SSP with non-identical parallel machines (SDSSP-NPM) model with total flow time as the main objective explicitly integrates three interdependent decisions: job sequencing, machine allocation, and tool switching scheduling. By jointly optimizing these decisions, the model is able to select job sequences that may involve more frequent tool changes but yield more efficient replacement combinations. This integrated optimization results in lower total setup time σ , reduced total flow time TFT, and a shorter makespan $F_{\max}$, as demonstrated in solution (b).

This example clearly illustrates that minimizing the number of tool switches alone is insufficient for achieving optimal production performance in sequence-dependent SSP environments. Instead, effective solutions must consider the interaction between job order, machine assignment, and tool replacement timing, particularly when setup times vary across tool pairs and machines.

4. Solution method

This section provides a comprehensive overview of the contributions made by this study. The two-stage heuristic presented in this study is adapted from Rifai et al. (2022a). In subsection 4.1, we introduce the ALNS heuristic, which focuses on determining the optimal job sequence and job allocation to the machine. Through iterative processes, the newly generated solution, denoted as P , yields valuable information about the required tools for each job in the sequence in each machine $T_j^m, \forall j \in P, m \in M$. Subsequently, in subsection 4.2, we introduce a combination of KTNS and

SA to determine the most effective tool switches, thus minimizing the setup time. As the concerned problem considering the properties of non-identical parallel machines, novel solution representations, destroy, and repair operators of ALNS are proposed to address the problem.

Table 3. The example of sequence-dependent tool setup time in two machines.

Machine 1									
Tools	1	2	3	4	5	6	7	8	9
1	0	1	2	3	2	3	2	4	2
2	1	0	5	3	1	5	1	3	4
3	2	5	0	4	4	5	5	5	4
4	3	3	4	0	3	2	4	2	4
5	2	1	4	3	0	4	3	1	5
6	3	5	5	2	4	0	4	1	5
7	2	1	5	4	3	4	0	1	1
8	4	3	5	2	1	1	1	0	1
9	2	4	4	4	5	5	1	1	0

Machine 2									
Tools	1	2	3	4	5	6	7	8	9
1	0	2	2	4	4	3	4	4	3
2	2	0	1	2	1	4	3	3	3
3	2	1	0	2	5	5	1	5	5
4	4	2	2	0	5	4	3	5	2
5	4	1	5	5	0	2	1	5	3
6	3	4	5	4	2	0	1	1	5
7	4	3	1	3	1	1	0	5	3
8	4	3	5	5	5	1	5	0	5
9	3	3	5	2	3	5	3	5	0

4.1. Stages 1: Job sequencing using ALNS

Large Neighborhood Search (LNS), originally proposed by Shaw (1998), is a metaheuristic designed to solve complex and highly constrained optimization problems by exploring extended neighborhoods. By partially destroying and reconstructing solutions, LNS mitigates the risk of entrapment in local optima and enhances global search capability (Pisinger & Ropke, 2010). Subsequently, Ropke and Pisinger (2006) introduced Adaptive Large Neighborhood Search (ALNS), which employs a performance-based learning mechanism to dynamically select destroy and repair operators. This adaptive strategy improves search efficiency by balancing intensification and diversification throughout the optimization process (Rifai et al., 2016).

In this study, we develop the ALNS method specifically for solving the job sequencing problem in the context of the sequence dependent SSP with non-identical parallel machines. ALNS serves as the higher-level solution method for job sequencing

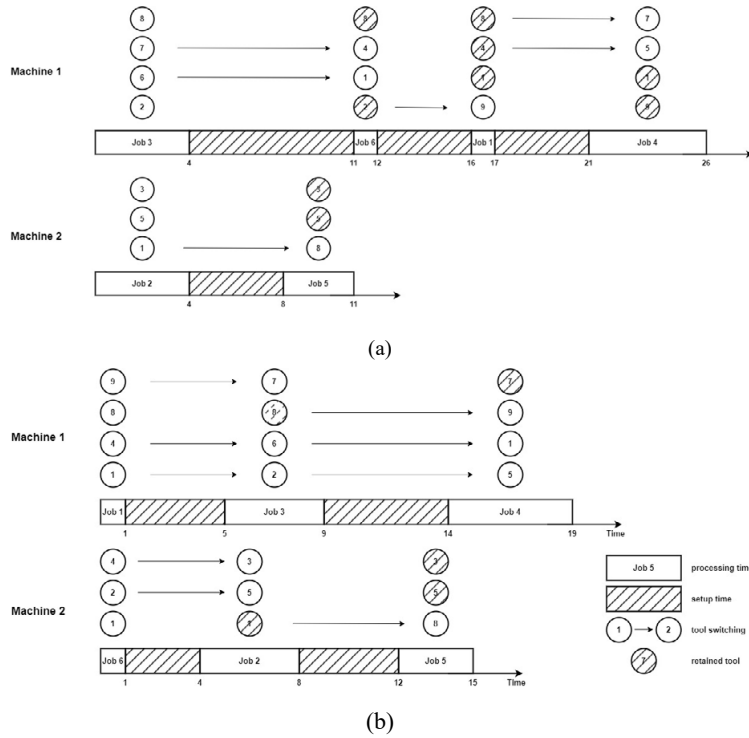


Figure 1. Examples of solutions: (a) minimizing the number of tool switches, (b) minimizing the total flow time

subproblem with two aims, allocating the jobs to machines and determining the sequence of jobs in each machine. Meanwhile, KTNS+SA serves as lower-level solution method for tool switching subproblem.

In each iteration, the job sequence and job allocation generated by ALNS become the input for the KTNS+SA method to solve the tool switching subproblem. Since KTNS+SA itself is also a metaheuristic that requires significant processing time, it is crucial for ALNS to achieve progressive improvements at each iteration and converge as early as possible without falling into premature convergence.

This requirement highlights the importance of maintaining an appropriate balance between exploration and exploitation throughout the search process. During the early iterations, stronger exploration is necessary to diversify the search space, allowing ALNS to investigate a wide range of job sequences and allocation patterns and avoid being trapped in poor-quality local optima. As the search progresses, the algorithm should gradually shift toward exploitation, intensifying the search around promising regions identified earlier. Algorithm 1 outlines the structure of the ALNS method developed for the job sequencing subproblem.

Algorithm 1: ALNS for job allocation and sequencing

```

1  input: an instance  $(J, T, M, \tau, T_j, C^m, p_j^m)$ ,
   ALNS parameters  $(\Gamma_1, \Gamma_0, \alpha, K, \lambda, \Omega, Q)$ 
2  initialize a feasible solution  $P$  randomly
3  perform KTNS+SA for tool switching
4  calculate the objective  $\sigma$ 
5  set  $P^* \leftarrow P, \sigma^* \leftarrow \sigma$ 
6  insert  $P$  to archive  $A$ 
7  initialize operator weights  $\omega_d^-$  and  $\omega_r^+$ 
8  while  $\Gamma > \Gamma_f$  do
9    select the destroy  $D^-$  and repair  $D^+$  adaptively
10   apply  $(D^-, D^+)$  to yield  $P'$ 
11   perform KTNS+SA for tool switching
12   calculate the objective  $\sigma'$ 
13    $\psi = \exp\left(\frac{-(\sigma' - \sigma)}{K(\Gamma)}\right)$ 
14   if  $\sigma' \leq \sigma$ 
15     if  $\sigma' \leq \sigma^*$ 
16        $P^* \leftarrow P', \sigma^* \leftarrow \sigma'$ 
17       insert  $P'$  to  $A$ 
18        $P \leftarrow P', \sigma \leftarrow \sigma'$ 
19        $q = U(0,1)$ 
20       if  $q < \psi$ 
21          $P \leftarrow P', \sigma \leftarrow \sigma'$ 
22     update  $\omega_d^-$  and  $\omega_r^+$ 
23     update  $\Gamma_{x+1} \leftarrow \alpha \Gamma_x$ 
24   output:  $P^*, \sigma^*$ 

```

The algorithm begins by inputting all required parameters of the instance ($J, T, M, \tau, \mathcal{T}_j, C^m, p_j^m$) and setting of ALNS parameters (initial temperature Γ_1 , final temperature Γ_0 , cooling rate α , Boltzman constant K , decay parameter λ , beta parameter Ω , and degree of destruction Q). The initial temperature Γ_1 , final temperature Γ_0 , cooling rate α , and Boltzman constant K regulate the iteration process and solution acceptance according to the Metropolis criterion. Decay parameter λ and weight adjustment parameter Ω are utilized for adaptive mechanism to update the weights of ALNS operators based on their historical performance (Ropke & Pisinger, 2006). Meanwhile, the degree of destruction Q determine how much portion (how many nodes) of a solution to be destroyed.

Afterward, an initial feasible solution P is randomly generated. The solution representation of the proposed ALNS for job sequencing subproblem is presented in Figure 2. The solution representation consists of two rows. The first row determines the sequence of jobs to be processed, while the second row is for machine assignment. As such, this representation is translated as job sequence 1–3–4 for machine 1 and 6–2–5 for machine 2. To optimize the tool switching and minimize the total setup time σ , we employ the KTNS+SA method. Once the objective (setup time, total flow time, makespan) are calculated, the initial solution is saved in the archive A .

Jobs	1	6	3	2	5	4
Machine	1	2	1	2	2	1

Figure 2. The solution representation for job sequencing subproblem.

In the next step, the iterations to generate new solutions are executed. The ALNS approach utilizes two types of operators: destroy (D^-) and repair (D^+) operators. Each operator consists of several mechanism with weights represented as ω_d^- and ω_r^+ for destroy and repair mechanism, respectively. The weights determine the probabilities of selecting each method during each iteration. This study uses roulette wheel to select the destroy and repair mechanism.

During each iteration, the solution P is modified by applying the selected destroy (D^-) and repair (D^+) operators to create a modified solution P' . The proposed ALNS framework incorporates multiple destroy and repair operators to strike a balance between intensifying and diversifying the search process. This study adapts some operators

from Rifai et al. (2022a), namely random removal, worst removal, worst removal without the 1st job, pair worst removal for destroy process, and random insertion, archived based insertion, elitist based insertion, fewest tool change insertion, reverse fewest tool change insertion for repair process. However, since there are two decision variables to be determined, job sequence and machine assignment, as illustrated in Figure 2, this study further develops three mechanisms for solution modification, which are job sequence only modification, machine assignment only modification, and simultaneous modification. Figure 3 presents the illustration of the mechanisms.

Destroy						Repair						Adjustment					
1	0	3	2	5	4	1	4	3	2	5	4	1	4	3	2	5	6
1	2	1	2	2	1	1	2	1	2	2	1	1	2	1	2	2	1
(a) sequence only modification																	
1	6	3	2	5	4	1	6	3	2	5	4	1	6	3	2	5	4
1	0	1	2	2	1	1	1	1	2	2	1	1	1	1	2	2	1
(b) machine assignment only modification																	
1	0	3	2	5	4	1	4	3	2	5	4	1	4	3	2	5	6
1	0	1	2	2	1	1	1	1	2	2	1	1	1	1	2	2	1
(c) simultaneous modification																	

Figure 3. The illustration of the three modification mechanisms.

The example in Figure 3 uses initial solution depicted in Figure 2. Random removal and random insertion are used as modification operators. The destroyed and repair nodes are highlighted. Noted that, after repair process, the generated solutions of mechanism (a) and (c) are infeasible due there are redundant value of jobs in the first row. As such, adjustments are performed to the other nodes. Although using the same destroy and repair operators, the three mechanisms result on different newly generated solutions. The mechanism (a) yields the job sequence of $M1: 1-3-6$; $M2: 4-2-5$, mechanism (b) yields $M1: 1-6-3$; $M2: 2-5$, and mechanism (c) yields $M1: 1-4-3$; $M2: 4-2-5$. The use of these three mechanisms adds greater flexibility of the solution modifications. Therefore, in the real implementation, there are 4 (type of operators) $\times$ 3 = 12 destroy operators, and 5 (type of operators) $\times$ 3 = 15 repair operators.

Subsequently, the KTNS+SA method is applied to the modified solution P' to determine the optimum tool switching schedule. Afterwards, the objective values of the modified solution P' are calculated.

The acceptance of the newly generated solution is decided by using the Metropolis criterion, which also directs the update of each operator weight. This adaptive mechanism ensures that the choice of destroy and repair methods is dynamically based on their past performance. The iteration continues until the current temperature reaches the final temperature.

4.2. Stages 2: Tool switching using KTNS+SA

KTNS stands for Keeping the Nearest Tools Soonest, an effective heuristic technique established by [Tang and Denardo \(1988\)](#), which offers a deterministic polynomial-time solution to the tooling problem. This technique has demonstrated its efficacy in solving the tooling problem within a reasonable timeframe.

The time complexity associated with solving the Tooling Problem (TP) in the context of a uniform Single-Stage Production (SSP) using KTNS is $O(MN)$, where M symbolizes the quantity of tools and N represents the number of jobs. The core concept underlying this approach is aptly conveyed by its name: the strategy centers around maintaining the earliest required tools in the magazine. This approach can be formally described through two straightforward procedures:

- During intermediate machining intervals, exclusively the tools necessary for processing the ongoing job are installed. There is no need to account for future tool requirements during this phase.
- Whenever a tool switch occurs, the tools present in the magazine are always the ones that will be needed most imminently.

KTNS's effectiveness has been substantiated in effectively addressing uniform tool switching scenarios, wherein it is assumed that the setup time of tools remains constant, irrespective of the preceding tool occupying the same slot. In such scenarios, determining the removal and installation of tools within any given slot can be straightforwardly calculated as they share identical costs.

However, when sequence-dependent tool switching is considered, a substantially more intricate decision-making process is required to identify the optimal tool replacement strategy. In such environments, the setup time is no longer solely determined by the presence or absence of a tool in the magazine, but also by the specific pair of tools involved in each replacement.

Consequently, two tool switching schedules with the same number of switches may result in significantly different total setup times depending on the order in which tools are removed and installed.

Algorithm 2: KTNS+SA for tool switching

```

1  input: an instance  $(J, P, T, \tau, \varphi)$ , SA parameters
    $(\Gamma_1, \Gamma_0, \alpha, K)$ 
2  for  $j = 1 : N$  do
3    if  $j = 1$ 
4      determine  $\gamma_1 \in T_1$ 
5    else
6      determine  $\gamma_j \in T_j$  and  $\theta_j$  using KTNS
7    set  $\gamma_j \rightarrow X$  for  $j \in J$  as input of SA
8    calculate  $\sigma(X)$ 
9    while stopping criterion is not reached do
10      $r = U(0,1)$ 
11      $[i, o] = U\{1, \beta_j\}$  for  $i \leq o$ 
12     if  $r \leq 1/3$  do swap move
13        $Y = \{X_1: X_{i-1}, X_o, X_{i+1}: X_{o-1}, X_i, X_{o+1}: X_{\beta_j}\}$ 
14     elseif  $1/3 < r \leq 2/3$  do insertion move
15        $Y = \{X_1: X_{i-1},$ 
16          $X_{i+1}: X_{o-1}, X_i, X_o, X_{o+1}: X_{\beta_j}\}$ 
17     elseif  $2/3 < r \leq 1$  do 2-opt move
18        $Z = \text{reverse}\{X_i: X_o\}$ 
19        $Y = \{X_1: X_{i-1}, Z, X_{o+1}: X_{\beta_j}\}$ 
20     calculate  $\sigma(Y)$ 
21      $\psi = \exp\left(\frac{-(\sigma(Y) - \sigma(X))}{K(\Gamma)}\right)$ 
22      $q = U(0,1)$ 
23     if  $\sigma(Y) \leq \sigma(X)$  or  $q < \psi$ 
24        $\gamma_j' \leftarrow Y; X \leftarrow Y; \sigma(X) \leftarrow \sigma(Y)$ 
25   output:  $\gamma_j', \sigma$ 
```

This fundamental difference renders classical KTNS insufficient for sequence-dependent SSPs. Although KTNS remains effective in determining which tools should be kept in the magazine to minimize the number of switches, it does not explicitly account for the cost heterogeneity among tool replacement pairs. As a result, KTNS alone may generate tool sets that are optimal in terms of switch count but suboptimal with respect to total setup time.

To address this limitation, a hybrid KTNS and simulated annealing (SA) approach is employed to determine the exact tool switching schedule under sequence-dependent setup times. The combined KTNS+SA method is adopted from [Rifai et al. \(2022a\)](#), in which KTNS is used to provide a high-quality initial solution structure, while SA is utilized

to refine the switching sequence by explicitly minimizing the total setup time. Algorithm 2 presents the pseudocode of the KTNS+SA procedure.

The algorithm begins by inputting all required parameters of the instance (J, P, T, τ, φ) and setting of SA parameters $(\Gamma_1, \Gamma_0, \alpha, K)$. At first, the sets of tools to inserted γ_j and replaced θ_j , without the exact information of switching schedule, are determined by using the KTNS. The set of inserted tools γ_j is then set as the solution to be determined in the SA stage. The solution modification is performed by randomly selecting two tool indices $\{i, o\}$ of the set γ_j . Afterward the tools at and in-between both indices are modified using three operators: swap move, insertion move, and 2-opt move. Each operator is given uniform weight so that it has equal probabilities to be selected in each iteration.

The fitness evaluation in the tool switching stage is based on minimizing the total setup time σ , which directly reflects the sequence-dependent switching cost incurred by the candidate solution. The acceptance of newly generated solutions is governed by the Metropolis criterion. Under this mechanism, solutions with inferior objective values may still be accepted during the early iterations, thereby enabling diversification and preventing the algorithm from becoming trapped in poor local optima.

As the temperature decreases, the probability of accepting worse solutions diminishes, causing the search behavior to gradually transition from exploration to exploitation. In the early phase, the algorithm emphasizes global exploration of the solution space, allowing diverse tool switching sequences to be evaluated. In the later phase, the algorithm increasingly concentrates on intensifying the search around high-quality regions, refining the tool replacement order to achieve minimal setup time.

The algorithm then terminates when either the current temperature Γ reaches the final temperature Γ_0 or the iteration process does not provide improvement over L_Y consecutive temperature reductions.

5. Computational experiment

In this segment, we outline the assessment of our suggested approach designed for the SDSSP involving non-identical parallel machines. First, the datasets used for the experiment are described. Since there is no study concerning SDSSP with non-identical parallel machine, the proposed method is evaluated in uniform SSP environment to showcase its initial performance. Afterward, the proposed two stage heuristic of ALNS and KTNS-SA (for SP and TP, consecutively) are compared with LNS and KTNS. Sensitivity analysis is then performed to provide managerial insight and to further evaluate the robustness of ALNS and KTNS-SA in delivering near-optimal solutions under various scenarios with different problem parameter values.

The algorithms were implemented and run within MATLAB R2019a on a computer equipped with an Intel® Core™ i7 CPU clocked at 2.90 GHz and 32GB of RAM. It is worth emphasizing that all experimentation was consistently conducted on the same computer to ensure an equitable comparison among all methodologies.

5.1. Experimental designs

We considered the datasets described [Calmels \(2022\)](#) to test the performance of our proposed method. The datasets consist of two main categories, SSP-NPM-I and SSP-NPM-II, both corresponding to small and large instances consecutively. Each instance has a specific number of machines M , number of jobs J , available tools T , magazine capacity of each machine C_m , tool switching time in each machine Sw_m , and job-tool matrix density. [Table 4](#) presents the characteristics of the instances for experiments.

There are a total of 640 instances with 160 belong to SSP-NPM-I (small instances) and 480 belong to SSP-NPM-II (large instances). Since the datasets are dedicated for the uniform SSP, we randomly generated the sequence-dependent setup time τ_{tu}^m exemplified in [Table 1](#), according to a discrete

Table 4. Characteristics of the instances.

$1M$	J	T	C_m	Sw_m	Job – Tool matrix density	Number of instances
2	{10,15}	{10,15}	(5,7)	(2,4)	sparse	80
3	{15,20}	{15,20}	(7,10,13)	(2,3,4)	sparse	80
4	{40,60}	60	$(25+(m-1) \times 5)$	{low, high}	{sparse, dense}	160
4	{40,60}	120	$(25+(m-1) \times 5)$	{low, high}	{sparse, dense}	160
6	{80,120}	120	$(45+(m-1) \times 5)$	{low, high}	{sparse, dense}	160

Table 5. The parameter setting used in the experiments.

Parameter	Meaning	Value
ALNS		
Γ_1	Initial temperature	100
Γ_0	final temperature	1
α	cooling rate	0.995
K	Boltzmann constant	0.2
$\Omega_1, \Omega_2, \Omega_3, \Omega_4$	weight adjustment parameters	0.3, 0.15, 0.03, 0
λ	decay parameter	0.9
Q_1	initial destroy ratio	0.75
Q_0	final destroy ratio	0.25
SA		
Γ_1	Initial temperature	10
Γ_0	final temperature	1
α	cooling rate	0.95
K	Boltzmann constant	0.2
I_Γ	number of iterations at a temperature	10

uniform distribution to evaluate the performance of the model and algorithm in various condition of setup time.

The Table 5 presents the parameter setting of the ALNS for job sequencing and SA to assist KTNS for tool switching subproblem. The SA is used inside the ALNS framework during determination of exact tool switching schedule, thus creating a mechanism of a metaheuristic inside metaheuristics. Therefore, to keep the computation time to be short without sacrificing the performance, the initial temperature Γ_1 and cooling rate α of SA is set at considerably lower values. The same parameter values are also applied to the benchmark method of LNS to ensure the fairness of comparison. Table 5 presents the parameter values for ALNS and SA.

5.2. Experimental results

5.2.1. Evaluation in uniform SSP-NPM

In this subsection, we evaluate the performance of the proposed ALNS for job sequencing and KTNS+SA for tool switching by comparing them against recent methods proposed by Calmels (2022). Since no previous study has addressed the SDSSP with non-identical parallel machines under sequence-dependent setup times, the comparison is conducted within the uniform SSP environment to ensure methodological consistency and fairness.

The evaluations are executed in two instances: $M = 2, J = 10, T = 10$ with sparse job-tool matrix density and $M = 2, J = 10, T = 10$ with dense job-tool matrix density, denoted as 2/10/15/s and 2/10/15/d, respectively. These two instances allow the assessment of algorithm robustness under different levels of tool interaction complexity. The sparse instance represents cases where jobs require relatively few tools, while the dense instance reflects realistic manufacturing settings in which jobs share many common tools and tool competition becomes more severe.

The benchmark methods are algorithms proposed by Calmels (2022) for uniform SSP with non-identical parallel machines which are Iterated Local Search (ILS) with three variants: ILS with objective-based perturbation (ILS P.1), ILS with random perturbation (ILS P.2), and ILS with combined perturbation (P.3). Since there are three concerned objectives (number of switches s , total flow time TFT, and makespan Fmax), each algorithm is executed three times to minimize each objective. Tables 6-8 presents the result of comparison experiments of the proposed and benchmark algorithms with the objective of minimizing number of switches s , total flow time TFT, and makespan Fmax, respectively.

As implied by the experimental results, in the 2/10/15/s instance, no substantial differences are observed among the methods with respect to the minimized objective values. The only notable exception is ILS P.2 in the minimization of the number of switches, which performs inferiorly compared to the other algorithms. However, although the objective values are similar, the algorithms exhibit distinct behaviors with respect to the remaining performance indicators.

As reported in Tables 6 and 7, the proposed ALNS-KTNS+SA demonstrates favorable overall performance by achieving a lower total flow time of 191 and a lower number of switches of 6 when compared to the benchmark methods. This indicates that the proposed approach is able to identify more balanced trade-off solutions even when the primary objective value is comparable. On the other hand, in the makespan minimization scenario, the proposed algorithm yields a slightly higher TFT than the ILS-based methods, suggesting that ILS may be more aggressive in compressing completion times when setup interactions are relatively limited.

Table 6. The result of comparison with ILS (Calmels, 2022) by minimizing number of switches as the objective.

Algorithms	2/10/15/s				2/10/15/d			
	No. of switches*	TFT	Makespan	Time (s)	No. of switches	TFT	Makespan	Time (s)
ILS P.1 (s)	3	199	41	600.19	16	367	72	600.31
ILS P.2 (s)	4	192	50	600.12	12	502	97	600.33
ILS P.3 (s)	3	199	41	600.01	13	415	86	600.06
ALNS-KTNS (s)	3	200	41	5.83	11	390	87	9.99
ALNS-KTNS+SA (s)	3	191	41	28.46	11	373	87	76.84

Table 7. The result of comparison with ILS (Calmels, 2022) by minimizing TFT as the objective.

Algorithms	2/10/15/s				2/10/15/d			
	No. of switches	TFT*	Makespan	Time (s)	No. of switches	TFT	Makespan	Time (s)
ILS P.1 (TFT)	9	110	30	600.07	19	331	73	600.04
ILS P.2 (TFT)	9	110	30	600.17	15	330	84	600.08
ILS P.3 (TFT)	9	110	30	600.11	18	329	83	600.25
ALNS-KTNS (TFT)	9	110	30	5.62	17	317	80	8.21
ALNS-KTNS+SA (TFT)	6	110	31	31.05	15	316	84	83.94

In contrast, for the 2/10/15/d instance with denser job–tool relationships and higher tool competition, the advantages of the proposed ALNS–KTNS+SA become significantly more pronounced. The dense instance substantially increases the complexity of the tool switching problem, as more tools must be frequently replaced and the impact of sequence-dependent setup times becomes dominant.

In the scenarios of minimizing the total number of switches (Table 6) and minimizing the total flow time (Table 7), the proposed method consistently yields the lowest values among all algorithms. In the makespan minimization scenario, the makespan obtained by ALNS–KTNS+SA attains the jointly lowest value. More importantly, when considering secondary performance metrics beyond the main objective, the proposed method achieves relatively better and more stable results than the benchmark algorithms across all objectives.

These observations highlight the capability of the proposed framework to generate high-quality schedules that maintain strong performance consistency across multiple conflicting criteria. While ILS focuses primarily on improving a single objective through localized perturbations, the ALNS-based approach benefits from diversified destroy–repair mechanisms that enable broader exploration of the solution space, particularly under dense tool interaction conditions.

Furthermore, the experimental results indicate that the proposed method is never outperformed by the benchmark algorithms with respect to the minimized objective in any tested scenario. In addition, the proposed ALNS–KTNS+SA requires significantly shorter computation time to deliver competitive or superior solutions. This computational efficiency is attributed to two key factors: (i) the ability of ALNS to rapidly converge toward high-quality job sequences through adaptive operator selection, and

Table 8. The result of comparison with ILS (Calmels, 2022) by minimizing makespan as the objective.

Algorithms	2/10/15/s				2/10/15/d			
	No. of switches	TFT	Makespan*	Time (s)	No. of switches	TFT	Makespan	Time (s)
ILS P.1 (Fmax)	9	110	30	600.03	19	331	73	600.22
ILS P.2 (Fmax)	9	110	30	600.11	16	372	69	600.19
ILS P.3 (Fmax)	9	110	30	600.28	17	368	69	600.12
ALNS-KTNS (Fmax)	9	153	30	5.93	15	345	69	8.18
ALNS-KTNS+SA (Fmax)	9	129	30	32.65	15	345	69	83.56

Table 9. The result of comparison with optimal value in SDSSP-NPM by minimizing total setup time as the objective.

Instance				Gurobi		ALNS-KTNS+SA		ALNS-KTNS	
No.	<i>M</i>	<i>J</i>	<i>T</i>	σ	Time (s)	σ	Time (s)	σ	Time (s)
1	2	10	10	15	4035	16	5.34	15	2.54
2	2	10	15	25	14400*	25	8.19	25	3.10
3	2	15	10	21	14400*	13	7.28	16	4.12
4	2	15	15	42	14400*	28	13.66	36	6.82
5	3	15	15	237	14400*	17	4.57	16	3.41
6	3	15	20	367	14400*	34	12.64	35	6.12
7	3	20	15	441	14400*	27	7.23	18	5.85
8	3	20	20	470	14400*	40	16.79	55	5.98

Table 10. The result of comparison with optimal value in SDSSP-NPM by minimizing total flow time as the objective.

Instance				Gurobi		ALNS-KTNS+SA		ALNS-KTNS	
No.	<i>M</i>	<i>J</i>	<i>T</i>	TFT	Time (s)	TFT	Time (s)	TFT	Time (s)
1	2	10	10	193	5320	212	5.87	201	1.63
2	2	10	15	263	14400*	261	10.52	270	1.86
3	2	15	10	415	14400*	384	9.21	404	3.44
4	2	15	15	559	14400*	430	18.83	465	3.72
5	3	15	15	1448	14400*	378	9.66	385	2.69
6	3	15	20	1707	14400*	440	16.58	465	2.88
7	3	20	15	N/A	14400*	603	16.88	648	4.22
8	3	20	20	N/A	14400*	734	23.10	759	5.33

Table 11. The result of comparison with optimal value in SDSSP-NPM by minimizing makespan as the objective.

Instance				Gurobi		ALNS-KTNS+SA		ALNS-KTNS	
No.	<i>M</i>	<i>J</i>	<i>T</i>	Fmax	Time (s)	Fmax	Time (s)	Fmax	Time (s)
1	2	10	10	36	7639	42	5.91	40	1.98
2	2	10	15	45	14400*	46	8.08	49	1.98
3	2	15	10	48	14400*	46	8.04	50	3.39
4	2	15	15	102	14400*	58	16.03	67	3.90
5	3	15	15	264	14400*	43	8.91	43	2.72
6	3	15	20	350	14400*	50	15.84	57	3.28
7	3	20	15	N/A	14400*	51	14.51	47	4.49
8	3	20	20	N/A	14400*	60	23.03	72	5.33

(ii) the use of KTNS to restrict the search space of tool switching, allowing SA to focus exclusively on optimizing sequence-dependent replacement orders.

These findings demonstrate that the proposed method is particularly well suited for handling complex tool switching environments and provide strong justification for its application to the SDSSP with non-identical parallel machines under sequence-dependent setup times, which is investigated in the subsequent experimental sections.

5.2.2. Comparison in SDSSP-NPM with exact method

In this subsection, the proposed algorithms were tested on small instances of SDSSP-NPM by comparing

with the optimal values obtained by Gurobi 9.0.2 executed in Python 3.7. The computation time limit for Gurobi is set to 14,400 seconds (4 hours) for each instance. Each algorithm is executed three times, with each run dedicated to minimizing one of the three objectives defined in the mathematical model. Tables 9–11 present the comparison results between the proposed algorithms and the optimal values for minimizing total setup time σ , total flow time TFT, and makespan Fmax, respectively.

Within the four-hour computation limit, the Gurobi solver is only able to optimally solve the SDSSP-NPM up to Instance 1 with $M = 2$, $J = 10$, $T = 10$, which is consistent with the findings reported by [Mara et al. \(2023\)](#). However, the computational burden of the SDSSP-NPM is substantially higher than that of the single-machine SDSSP investigated in [Mara et al.](#)

(2023). This increase in complexity arises from the incorporation of multiple non-identical machines and the additional time-related constraints formulated in Equations (19)–(24), which explicitly model job assignment, machine availability, completion times, and inter-job dependencies.

Furthermore, the optimization models that minimize total flow time and makespan are considerably more complex than the setup-time minimization model. While minimizing setup time mainly focuses on tool replacement decisions, both TFT and makespan objectives are directly coupled with processing times, setup times, and job completion variables. As a result, these objectives significantly intensify the interdependencies among constraints and enlarge the branch-and-bound search tree explored by the solver.

Consequently, Gurobi 9.0.2 executed on the experimental device is unable to produce feasible solutions within the four-hour time limit for Instances 7 and 8, which are reported as N/A in the tables. This limitation highlights the rapid growth of model complexity even for moderately sized SDSSP-

NPM instances and confirms the computational intractability of exact approaches for practical-scale problems.

The comparison results further indicate that the proposed metaheuristic methods outperform the commercial solver starting from Instance 3 and onwards. In particular, consistently better results are obtained by the proposed ALNS–KTNS+SA compared to both Gurobi and the ALNS–KTNS approach across all three objective models as instance size increases. This performance gap becomes increasingly evident in larger instances, where the solver either fails to reach optimality or is unable to generate feasible solutions within the allocated time.

It is observed that ALNS–KTNS requires slightly lower computation time than ALNS–KTNS+SA, as it relies solely on KTNS to solve the tool switching subproblem and does not explicitly consider the impact of sequence-dependent setup times. While this simplified strategy leads to faster computation, it overlooks potential improvements achievable through optimized tool replacement ordering.

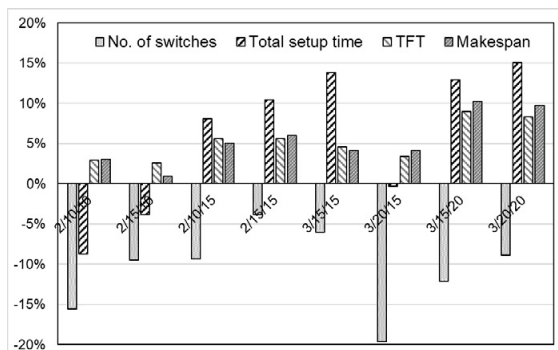


Figure 4. Improvement gap of best objective obtained by KTNS+SA over KTNS in small instances.

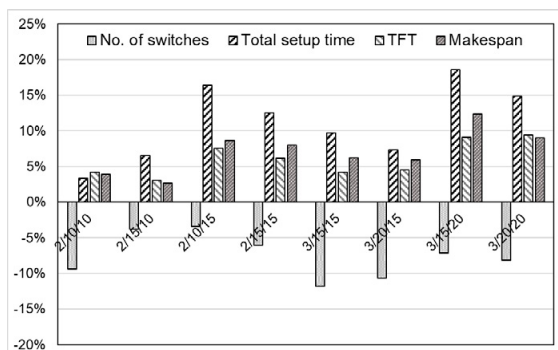


Figure 5. Improvement gap of average objective obtained by KTNS+SA over KTNS in small instances.

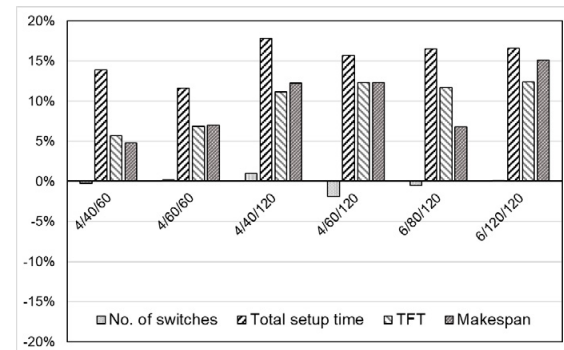


Figure 6. Improvement gap of best objective obtained by KTNS+SA over KTNS in large instances.

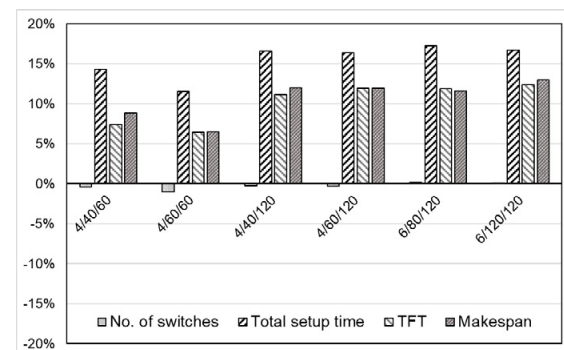


Figure 7. Improvement gap of average objective obtained by KTNS+SA over KTNS in large instances.

In contrast, the ALNS–KTNS+SA further explores the tool switching subproblem by searching for the optimal tool switching sequence using simulated annealing. Although this additional optimization layer incurs extra computational cost, it enables the algorithm to effectively capture the influence of sequence-dependent setup times. As a result, ALNS–KTNS+SA consistently achieves lower total setup time, reduced total flow time, and improved makespan compared to ALNS–KTNS, particularly in instances with dense job–tool relationships. These results clearly demonstrate that solution quality in SDSSP-NPM is highly sensitive not only to job sequencing and machine allocation decisions, but also to the precise ordering of tool replacements. Ignoring sequence dependency may yield faster solutions but leads to suboptimal schedules when setup-time heterogeneity becomes significant.

5.2.3. Analysis of tool switching method

This subsection discusses the evaluation of the significance of combined KTNS and SA for tool switching subproblem. The evaluation is conducted by comparing the combined KTNS+SA with the standard KTNS. ALNS with exactly the same parameter setting is used for job sequencing subproblem for both tool switching methods. The evaluation is executed by comparing the number of switches, total setup time, TFT, and makespan obtained by both methods.

To highlight the improvement obtained by the proposed KTNS+SA over the benchmark algorithm, the gap percentage Δ is used as relative performance measure. The gap improvement Δ is calculated as follow.

$$\Delta = \frac{AF_x - AF_y}{AF_x} \times 100\% \quad (24)$$

where AF_x is the objective of benchmark algorithm x , and AF_y is the fitness of the proposed algorithm y . The gap calculations of best objective Δ^* and average objective $\bar{\Delta}$ follow the same procedure in the Equation (24). The gaps between the algorithms on small instances for best and average objective are presented in Figures 4 and 5. Meanwhile, Figures 6 and 7 present the improvement gaps of best and average objective on large instances. Complete results of the experiments on all instances can be accessed in Appendix 1.

Based on the results obtained from small-scale instances, it is observed that there is a negative improvement in the number of switches, indicating that the combined KTNS+SA generally produces a greater number of tool switches than the standard KTNS. However, despite this increase, significant improvements are consistently achieved in total setup time, total flow time (TFT), and makespan for both the best and average objective values.

This phenomenon can be explained by the fundamental characteristics of the KTNS heuristic. KTNS is explicitly designed to minimize the number of tool switches by prioritizing the retention of tools that will be required soonest in the upcoming job sequence. As a result, its decision logic focuses primarily on reducing switch frequency rather than evaluating the cost associated with individual tool replacement pairs. Consequently, KTNS is indifferent to whether a specific replaced–installed tool pair leads to high or low setup time, even though setup times may vary substantially across different tool combinations.

In contrast, the combined KTNS+SA approach is capable of determining the exact tool switching schedule by explicitly evaluating the sequence-dependent setup times. Through the simulated annealing stage, the algorithm searches for favorable replacement orders among the candidate tool sets generated by KTNS, allowing it to identify optimal or near-optimal tool pair combinations. This optimization process may occasionally require additional tool switches; however, these additional switches are strategically selected to significantly reduce the cumulative setup time. As a result, even with a slightly higher number of switches, the total setup time is considerably reduced, which directly translates into substantial improvements in TFT and makespan.

In large-scale instances, the pattern of negative improvement in the number of switches is no longer observed. Specifically, no significant differences are found between KTNS+SA and the standard KTNS in terms of switch count for both best and average objective values. This behavior can be attributed to the increased structural complexity of larger instances, where the number of feasible tool combinations and switching alternatives grows rapidly. In such environments, minimizing the number of switches alone becomes insufficient to distinguish high-quality solutions, and both methods naturally converge toward similar switch counts.

Nevertheless, the improvement patterns observed in total setup time, TFT, and makespan remain consistent—and are in fact more pronounced—in larger instances. The magnitude of improvement increases with the number of machines M , the number of jobs J , and the number of available tools T . As instance size grows, the tool switching problem becomes exponentially more complex due to the rapidly expanding number of possible replaced–installed tool pair combinations and switching permutations.

Under these conditions, the advantage of the proposed KTNS+SA becomes increasingly evident. While the standard KTNS only determines which tools should be removed and installed—without considering the exact order or pairing—the KTNS+SA explicitly searches for the optimal replacement sequence among these tools. This capability enables the proposed method to exploit favorable tool pairings that yield lower setup times, which cannot be captured by switch-count–based heuristics.

Furthermore, the results reveal no substantial discrepancy between improvements obtained in best-case and average-case objectives. This observation partially highlights the robustness of the proposed approach, indicating that KTNS+SA consistently produces high-quality solutions across multiple independent runs rather than relying on occasional favorable random outcomes. Such consistency is particularly important for large-scale industrial applications, where solution reliability and repeatability are as critical as absolute optimality.

5.2.4. Analysis of job sequencing method

For the evaluation of the job sequencing method, the proposed Adaptive Large Neighborhood Search (ALNS) is compared against the standard Large Neighborhood Search (LNS) to assess the contribution of incorporating an adaptive mechanism for destroy and repair operator selection. In both algorithmic variants, exactly the same destroy and repair operators are employed, and identical parameter values are applied throughout the experiments. This design ensures that any observed performance difference can be attributed solely to the adaptive operator selection mechanism rather than to operator design or parameter tuning effects.

Regarding the tool switching component, both algorithms employ the same KTNS+SA procedure to determine the tool switching schedule. Consequently, the comparison exclusively reflects the effectiveness of the job sequencing strategy. The performance assessment focuses on total flow time (TFT) and makespan, which represent the primary optimization objectives in the SDSSP with non-identical parallel machines. Figures 8 and 9 illustrate the comparison of the obtained TFT and makespan between the proposed ALNS and standard LNS on small-scale instances, while Figures 10 and 11 present the corresponding results for large-scale instances.

The experimental results indicate that, overall, no substantial performance gap exists between the two job sequencing methods across all instances. However, a closer inspection reveals distinct patterns depending on problem size. On small instances, marginal negative improvements are observed, with average values of $\Delta_{TFT}^* = -1.36\%$, $\bar{\Delta}_{TFT} = -1.98\%$, $\Delta_{Fmax}^* = 0\%$, $\bar{\Delta}_{Fmax} = -1.17\%$.

These results indicate that the standard LNS slightly outperforms the ALNS in small-scale problems.

This behavior can be attributed to the limited solution space and relatively low combinatorial complexity of small instances. In such cases, most destroy–repair operators exhibit similar effectiveness, and the search landscape contains fewer deep local optima. As a result, uniform random selection of operators—as implemented in standard LNS—is often sufficient to identify high-quality solutions. Conversely, the adaptive learning mechanism of ALNS requires multiple iterations to accurately estimate operator performance. When the problem size is small, the search may converge before meaningful performance differentiation among operators can be established, thereby reducing the benefit of adaptivity.

In contrast, a reversed performance trend is observed in large-scale instances. In these cases, the proposed ALNS slightly outperforms LNS for both objectives, with average improvements of $\Delta_{TFT}^* = 0.17\%$, $\bar{\Delta}_{TFT} = 0\%$, $\Delta_{Fmax}^* = 0\%$, $\bar{\Delta}_{Fmax} = 0.29\%$. Although the magnitude of improvement appears modest, the consistent dominance across both TFT and makespan objectives indicates a systematic advantage of the adaptive mechanism under higher problem complexity.

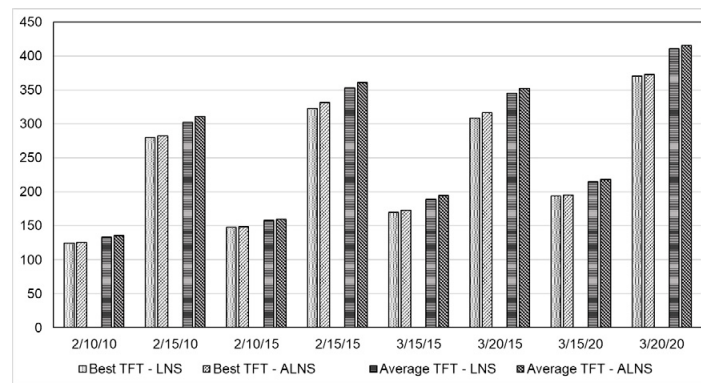


Figure 8. Comparison of Total Flow Time obtained by LNS and ALNS in small instances.

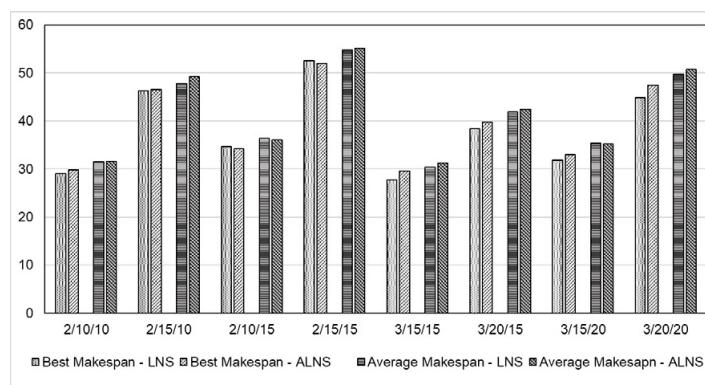


Figure 9. Comparison of Makespan obtained by LNS and ALNS in small instances.

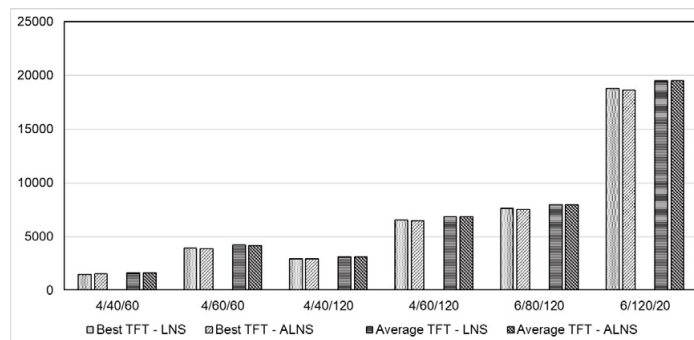


Figure 10. Comparison of Total Flow Time obtained by LNS and ALNS in large instances.

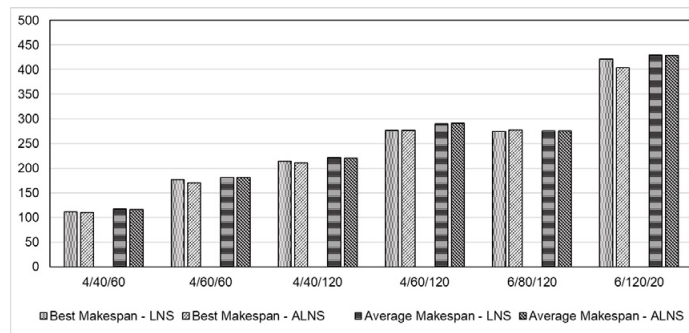


Figure 11. Comparison of Makespan obtained by LNS and ALNS in large instances.

The superior performance of ALNS in large instances can be explained by the increased diversity and heterogeneity of the search space. As the number of jobs, machines, and tools grows, the effectiveness of destroy and repair operators becomes highly instance-dependent. Certain operators perform better in early exploration stages, while others are more effective during solution refinement. The adaptive scoring mechanism of ALNS allows the algorithm to progressively favor operators that have demonstrated superior historical performance, thereby allocating more computational effort to promising neighborhoods.

This learning-based operator selection enables ALNS to dynamically balance exploration and exploitation throughout the search process—an advantage that becomes increasingly important as the combinatorial search space expands. In contrast, standard LNS lacks this capability, as all operators are selected with equal probability regardless of their observed contribution to solution improvement.

5.2.5. Sensitivity analysis

This section examines the effect of two important parameters in the SSP with non-identical parallel machines, which are the number of required tools for each job and the length of switching time Sw_m . There are two levels of the length of switching time Sw_m , low and high. The switching time values are calculated relative to the length of processing time of each job in certain machine p_j^m . This parameter values are generated using uniform distribution according to the rule $Sw_m = U(0.5 \times p_j^m, 1.0 \times p_j^m)$ for low level value of switching time, and $Sw_m = U(0.75 \times p_j^m, 1.25 \times p_j^m)$ for high level value of switching time. In the context of SDSSP, the sequence-dependent switching/setup times are generated exclusively for each pair of removed-installed tools in each machine. The sensitivity analysis is performed only on the large datasets to evaluate the impact of this parameter on the data with high complexity. Figure 12 presents the results of sensitivity analysis on the two levels of switching time Sw_m .

The results indicate that the increase on tool switching time affects the increase of total setup time, TFT, and makespan with average increment of 39.41%, 31.5% and 33.1%, respectively. The increase is relatively uniform in all instances size, regardless of the complexity. It indicates that the increase of the objectives is solely influenced by

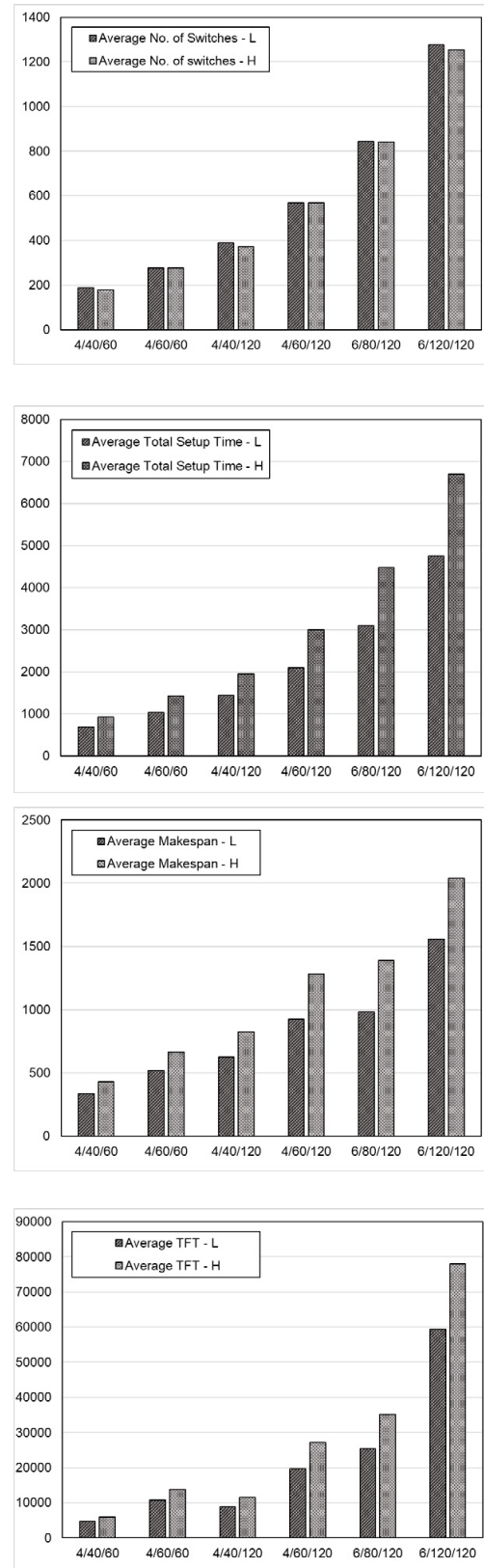


Figure 12. The results of sensitivity analysis on the two levels of switching time Sw_m (Low vs. High).



longer tool switching time, without alterations on job processing sequence, machine allocation, and tool switching schedule. This claim is reinforced by the results on the number of tool switches which are relatively the same between the two scenarios, which indicate that the obtained solutions on both scenarios are similar.

Moving on, sensitivity analysis is conducted to evaluate the influence of the number of required tools for each job $\mathcal{T}_j$ on the objectives obtained by the proposed ALNS and KTNS+SA method. The number of required tools for each job $\mathcal{T}_j$ are represented as job-tools matrix density with two categories, sparse (S) and dense (D). This metrics represents the number of required tools to process a job relative to the tool magazine capacity of each machine C_m . The values are randomly generated using uniform distribution with the formulation of $\mathcal{T}_j = U\left(\min\left[\frac{C_m}{4}\right], \min\left[\frac{C_m}{2}\right]\right)$ for sparse and $\mathcal{T}_j = U\left(\max\left[\frac{C_m}{2}\right], \max\left(C_m\right)\right)$ for dense (Calmels, 2022). Similar to the previous analysis, the effect of this parameter is evaluated on high complexity instances. Figure 13 presents the results of sensitivity analysis on the two levels of number of required tools $\mathcal{T}_j$.

From the results of sensitivity analysis, it is observed that the increase on the number of required tools significantly increases the average of total setup time, TFT, and makespan at 440.71%, 410.62% and 457.24%, respectively. Unlike in the previous sensitivity analysis, the change on job-tool matrix density drastically alters the switching schedule which eventually might change the job sequence and machine allocation. This observation is supported by the behavior of the model, in which there is also significant increase on the number of switches with average of 442.31% due to larger number of tools to be replaced and installed tools even though having the same number of jobs. The increases in the number of switches, total setup time, TFT, and makespan are mostly consistent in every instance, except for a slight decrease on the TFT on 4/40/120 instances as compared to 4/60/60 instances. The decrease is mostly attributed to other factors, such as the difference in the processing time and the lower number of jobs.

Further, we also evaluate the effectiveness of the proposed KTNS+SA for the tool switching method as compared to the standard KTNS in each scenario. Figures 14 and 15 present the improvement gap on each indicator obtained by the proposed KTNS+SA

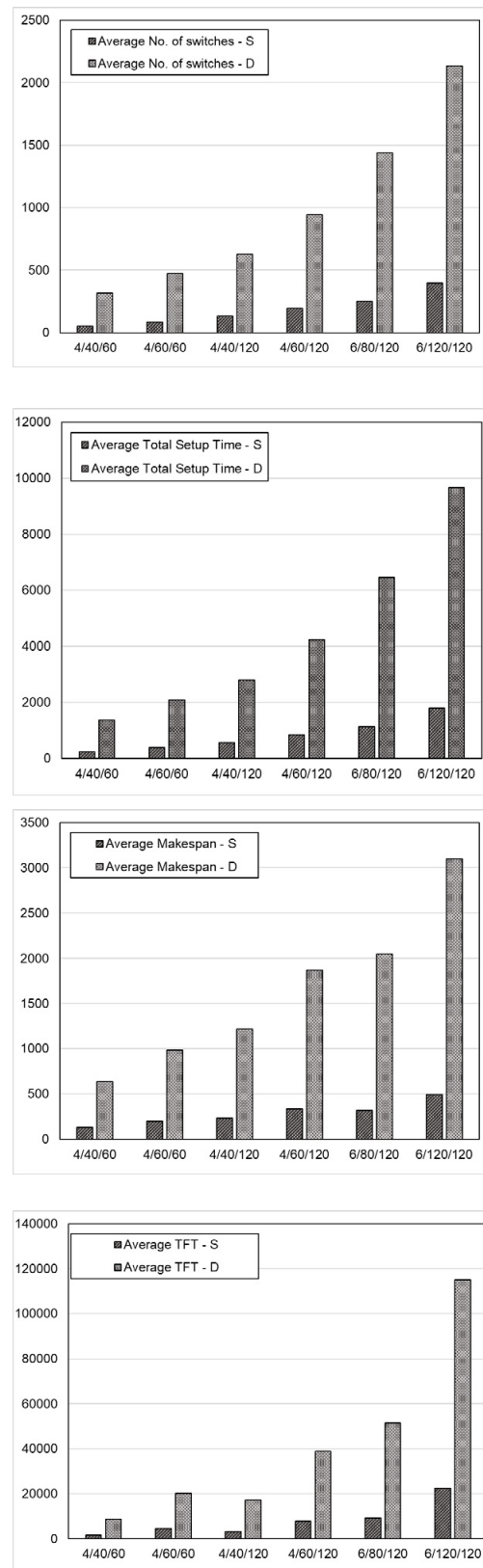


Figure 13. The results of sensitivity analysis on the two levels of number of required tools $\mathcal{T}_j$ (Sparse vs. Dense).

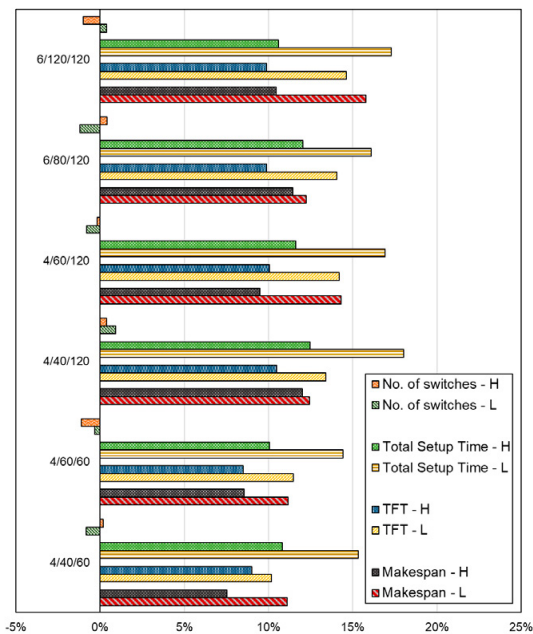


Figure 14. Improvement gap of best objective obtained by KTNS+SA over KTNS in two levels of the switching time (L vs. H).

against the standard KTNS in two levels of the switching time (L vs. H) and in two levels of the number of required tools (S vs. D), respectively.

As observed in Figures 14-15, the use of proposed KTNS+SA improves the method in obtaining lower objectives (total setup time, TFT and makespan) by a considerable margin in all scenarios. Meanwhile, the use of KTNS+SA does not significantly change the number of switches in all scenarios, consistent with the result in subsection 5.2.2. In the analysis on the tool switching time, the improvement gaps of KTNS+SA over standard KTNS on the three objectives are higher on the instances with low switching time. In high switching time level, the contribution of the increase on the switching time to the total setup time, TFT and makespan is much higher than the importance of finding optimal switching schedules, as also observed in the previous analysis.

In the analysis on the number of required tools, the proposed KTNS+SA yields higher improvement gaps on the instances with denser job-tool matrix density. This result means that the impact of KTNS+SA in generating near-optimal switching schedules has linear relationships with the number of required tools. It is understandable since in the later scenario, there are higher numbers of tool switches that need to be optimized its switching schedule. Hence,

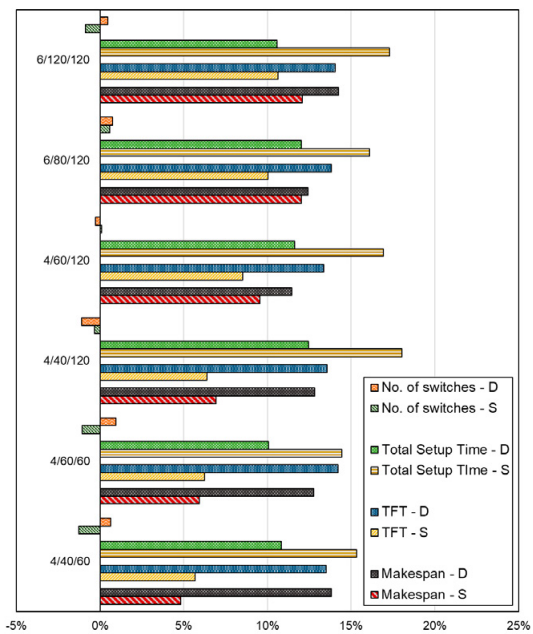


Figure 15. Improvement gap of best objective obtained by KTNS+SA over KTNS in two levels of the number of required tools (S vs. D).

the sensitivity analysis also shows the favorable performance of the proposed KTNS+SA in complex data with larger number of tools to be switched.

5.2.6. Analysis of computing time

At the last subsection, we evaluate the efficiency of the methods in generating solutions by comparing average computation time of both LNS and ALNS for job sequencing, and KTNS and KTNS+SA for tool switching. Table 12 presents the average computation time of all methods over ten replications and ten instances in each data configuration.

The results presented in Table 12 indicate that, for the tool switching method, the incorporation of SA leads to a noticeable increase in computation time. This behavior is expected, as SA is an iterative metaheuristic that performs repeated neighborhood exploration to obtain a near-optimal switching schedule. The evaluation of multiple candidate solutions at different temperature levels, combined with probabilistic acceptance of inferior solutions during early iterations, requires a considerable amount of computational effort.

Nevertheless, this additional computation time is a direct consequence of the enhanced solution quality achieved by KTNS+SA, particularly in

Table 12. Average computation time (s) of all methods.

M	J	T	LNS		ALNS	
			KTNS	KTNS+SA	KTNS	KTNS+SA
2	10	10	1.89	4.54	1.85	4.43
2	15	10	3.89	8.41	3.70	8.05
2	10	15	2.18	6.81	2.11	6.60
2	15	15	4.08	11.29	3.97	10.98
3	15	15	2.77	6.55	2.66	6.35
3	20	15	4.41	9.70	4.19	9.49
3	15	20	3.22	9.52	3.08	9.40
3	20	20	6.20	17.19	6.06	17.16
4	40	60	17.57	53.56	16.50	53.35
4	60	60	38.84	85.87	36.75	85.24
4	40	120	17.50	62.65	16.46	62.56
4	60	120	39.48	137.30	37.52	137.68
6	80	120	52.69	141.50	48.93	141.45
6	120	120	119.49	344.77	111.61	343.73

environments with sequence-dependent setup times. While the standard KTNS determines only the set of tools to be inserted and removed, the SA stage further optimizes the exact replacement order, which substantially enlarges the search space but enables significant reductions in total setup time, total flow time, and makespan.

In contrast, for the job sequencing method, no significant difference is observed between the computation times of LNS and ALNS. Both approaches utilize identical parameter settings and the same destroy and repair operators, and therefore exhibit comparable computational complexity per iteration. This observation confirms that the introduction of the adaptive mechanism for selecting destroy and repair operators does not impose additional computational overhead. The adaptivity

in ALNS is implemented solely through lightweight scoring and probability-updating procedures, which contribute negligibly to the total runtime when compared with solution construction and evaluation processes.

In addition to the algorithm-level comparison, a comprehensive computation time evaluation of the proposed ALNS for job sequencing and KTNS+SA for tool switching is conducted across all scenarios of tool switching time and number of required tools, as illustrated in [Figure 16](#).

The results show that there is no evident correlation between longer tool switching time values and increased computation time. This outcome is expected, as the magnitude of setup time parameters affects only the objective function value

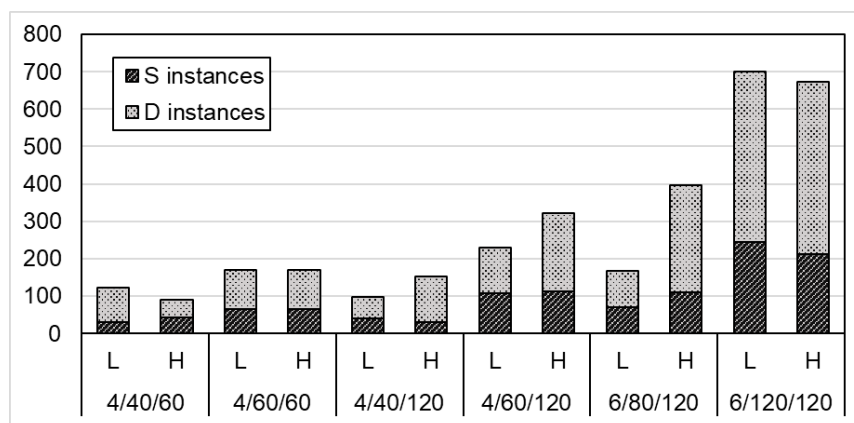


Figure 16. Computation time of the proposed method over two levels of tool switching time (L vs. H) and number of required tools (S vs. D).

and does not alter the combinatorial structure of the optimization problem. Consequently, longer or shorter switching times do not change the number of feasible schedules or neighborhood size explored by the algorithms.

In contrast, a clear increase in computation time is observed with the growth in the number of required tools. This parameter directly influences problem complexity by expanding the number of feasible tool combinations and possible replacement sequences. As the tool set becomes larger, the number of candidate replaced–installed tool pairs increases combinatorially, resulting in a substantially larger search space.

The rise in computation time is therefore mainly attributed to the KTNS+SA procedure, which must evaluate a greater number of tool switching permutations to identify high-quality schedules. With more tools involved, the SA component requires additional iterations to sufficiently explore the expanded neighborhood space, leading to longer convergence times. This behavior highlights that the computational burden of the proposed framework is primarily driven by the complexity of the tool switching subproblem rather than by job sequencing.

5.3. Discussion

The SDSSP-NPM is inherently complex due to several factors. First, this problem considers sequence-dependent setup time for tool switching, which requires finding the optimal tool switching schedule. This tool switching schedule details the pair of removed and installed tools on certain magazine slot. Meanwhile, in uniform SSP, this detail is unnecessary since the installed tools can be simply positioned in any slot. Second, the consideration of non-identical parallel machines adds to the decision of optimal machine assignments. Different machines have varying processing capabilities, tool magazine capacities, and processing times, making the job allocation, and sequencing more complicated since it is necessary to determine the optimal machine assignment and the job sequence in each machine. As such, the problem is combinatorial, meaning the number of possible solutions grows exponentially with the number of jobs, machines, and tools, making it computationally intensive to find optimal solutions.

To solve this problem, this study proposed a decomposition approach, where the problem is decomposed into two subproblems: job sequencing and tool switching. This decomposition is strategic and based on several practical reasons. First, the decomposition technique aims to reduce complexity. By breaking down the problem into smaller, more manageable subproblems, the complexity of each subproblem is significantly reduced. This allows for more focused and efficient solution approaches for each part. Second, different subproblems often benefit from different specialized solution techniques. Based on the experimental results, the use of ALNS can effectively address job sequencing subproblem, while the combination of KTNS+SA is efficient on solving the tool switching subproblem. Combining these techniques directly into one large problem might dilute their effectiveness.

Solving the problem in stages can be computationally more efficient. Each stage can be optimized separately, leading to faster convergence, and reduced overall computation time compared to solving the entire problem simultaneously. Combining the two subproblems into a single large optimization problem increases the computational load exponentially. The solution space becomes larger and more complex, making it harder to find optimal solutions within reasonable timeframes, as indicated in subsection 5.2.2. The integrated approach using exact solver Gurobi based on the proposed mathematical model can only optimally solve the smallest instance within the limit of four hours computation time.

Further, decomposing the problem makes the approach more scalable. The interdependence between job sequencing and tool switching introduces additional layers of complexity. Changes in job sequences affect tool switching schedules and vice versa, complicating the optimization process. As the number of jobs, tools, and machines increases, the solution methods can be adapted and scaled more easily when applied to smaller subproblems. In addition, decomposing the problem aligns the solution approach with practical implementation and operational workflows. In real-world manufacturing, job sequencing and tool switching are often handled by different systems or stages in the production process. Based on these reasons, the proposed decomposition approach of ANLS-KTNS+SA provides some benefits for solving the SDSSP-NPM.

6. Conclusion

This research addresses the sequence-dependent job scheduling and tool switching problem in environments with non-identical parallel machines. Such problems commonly arise in manufacturing systems, including metalworking and semiconductor production, where different jobs require distinct sets of tools for processing. The study develops a Mixed Integer Linear Programming (MILP) model based on a five-index formulation to represent the SDSSP-NPM. The proposed formulation simultaneously considers three optimization objectives: minimizing total flow time, minimizing makespan, and minimizing total setup time.

As a solving method, a two-stage metaheuristic based on ALNS and combination of SA and KTNS is proposed to solve the job sequencing and tool switching subproblems, respectively. A set of experiments on 640 instances with various complexities is executed to evaluate the proposed method. First, comparisons against three variants of iterated local search methods (Calmels, 2022) are conducted in uniform SSP with non-identical parallel machine. Afterward, analysis of combined SA and

KTNS against the standard KTNS is performed in the tool switching subproblem. Subsequently, the performance of proposed ALNS for solving the job sequencing subproblem is evaluated against the standard LNS as benchmark method. The results of the experiment indicate that the proposed method performs favorably over all benchmark methods both in uniform and sequence dependent SSP with non-identical machines.

Further, sensitivity analysis is conducted to examine the effect of two important parameters in the SDSSP-NPM, which are the number of required tools for each job and the length of switching time. The results confirm the ability of the proposed MOALNS in handling the changing value of these two parameters, indicated by higher improvement gaps on the instances with denser job-tool matrix density. Therefore, the results show the applicability of the proposed method in any scenario. Finally, future research can be performed by considering multi-objective scenarios of the problem, since there is indication that the three objectives can have trade-off relationships in certain conditions. The model can also be expanded by relaxing some assumptions, such as non-uniform tool size and unequal compatibility of magazine slot.

References

- Adjiaşvili, D., Bosio, S., & Zemmer, K. (2015). Minimizing the number of switch instances on a flexible machine in polynomial time. *Operations Research Letters*, 43(3), 317–322. <https://doi.org/10.1016/j.orl.2015.04.001>
- Atta, S., Sinha Mahapatra, P. R., & Mukhopadhyay, A. (2019). Solving tool indexing problem using harmony search algorithm with harmony refinement. *Soft Computing*, 23, 7407–7423. <https://doi.org/10.1007/s00500-018-3385-5>
- Beezão, A. C., Cordeau, J.-F., Laporte, G., & Ynanse, H. H. (2017). Scheduling identical parallel machines with tooling constraints. *European Journal of Operational Research*, 257(3), 834–844. <https://doi.org/10.1016/j.ejor.2016.08.008>
- Burger, A. P., Jacobs, C. G., van Vuuren, J. H., & Visagie, S. E. (2015). Scheduling multi-colour print jobs with sequence-dependent setup times. *Journal of Scheduling*, 18, 131–145. <https://doi.org/10.1007/s10951-014-0400-2>
- Calmels, D. (2019). The job sequencing and tool switching problem: State-of-the-art literature review, classification, and trends. *International Journal of Production Research*, 57(15–16), 5005–5025. <https://doi.org/10.1080/00207543.2018.1505057>
- Calmels, D. (2022). An iterated local search procedure for the job sequencing and tool switching problem with non-identical parallel machines. *European Journal of Operational Research*, 297(1), 66–85. <https://doi.org/10.1016/j.ejor.2021.05.005>
- Catanzaro, D., Gouveia, L., & Labbé, M. (2015). Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3), 766–777. <https://doi.org/10.1016/j.ejor.2015.02.018>
- Chaves, A. A., Lorena, L. A. N., Senne, E. L. F., & Resende, M. G. C. (2016). Hybrid method with CS and BRKGA applied to the minimization of tool switches problem. *Computers & Operations Research*, 67, 174–183. <https://doi.org/10.1016/j.cor.2015.10.009>
- Cura, T. (2023). Hybridizing local searching with genetic algorithms for the job sequencing and tool switching problem with non-identical parallel machines. *Expert Systems with Applications*, 223, Article 119908. <https://doi.org/10.1016/j.eswa.2023.119908>

- Dang, Q.-V., van Diessen, T., Martagan, T., & Adan, I. (2021). A matheuristic for parallel machine scheduling with tool replacements. *European Journal of Operational Research*, 291(2), 640–660. <https://doi.org/10.1016/j.ejor.2020.09.050>
- Dang, Q.-V., Herps, K., Martagan, T., Adan, I., & Heinrich, J. (2023). Unsupervised parallel machines scheduling with tool switches. *Computers & Operations Research*, 160, Article 106361. <https://doi.org/10.1016/j.cor.2023.106361>
- Furrer, M., & Mütze, T. (2017). An algorithmic framework for tool switching problems with multiple objectives. *European Journal of Operational Research*, 259(3), 1003–1016. <https://doi.org/10.1016/j.ejor.2016.11.034>
- Ghrayeb, O. A., Phojanamongkolkij, N., & Finch, P. R. (2003). A mathematical model and heuristic procedure to schedule printed circuit packs on sequencers. *International Journal of Production Research*, 41(16), 3849–3860. <https://doi.org/10.1080/0020754031000118071>
- Gökgür, B., Hnich, B., & Özpeynirci, S. (2018). Parallel machine scheduling with tool loading: A constraint programming approach. *International Journal of Production Research*, 56(16), 5541–5557. <https://doi.org/10.1080/00207543.2017.1421781>
- Grassi, A., Guizzi, G., Popolo, V., & Vespoli, S. (2023). A genetic-algorithm-based approach for optimizing tool utilization and makespan in FMS scheduling. *Journal of Manufacturing and Materials Processing*, 7(2), Article 75. <https://doi.org/10.3390/jmmp7020075>
- Iori, M., Locatelli, A., Locatelli, M., & Salazar-González, J. J. (2022). Tool switching problems in the context of overlay printing with multiple colours. In *Combinatorial Optimization* (pp. 260–271). https://doi.org/10.1007/978-3-031-18530-4_19
- Javaid, M., Haleem, A., Singh, R. P., & Suman, R. (2022). Enabling flexible manufacturing system (FMS) through the applications of industry 4.0 technologies. *Internet of Things and Cyber-Physical Systems*, 2, 49–62. <https://doi.org/10.1016/j.iotcps.2022.05.005>
- Khan, B. K., Gupta, B. D., Sen Gupta, D. K., & Kumar, K. D. (2000). A generalized procedure for minimizing tool changeovers of two parallel and identical CNC machining centres. *Production Planning & Control*, 11(1), 62–72. <https://doi.org/10.1080/095372800232496>
- Laporte, G., Salazar-Gonzalez, J. J., & Semet, F. (2004). Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1), 37–45. <https://doi.org/10.1080/07408170490257871>
- Mara, S. T. W., Sutoyo, E., Norcahyo, R., & Rifai, A. P. (2023). The job sequencing and tool switching problem with sequence-dependent setup time. *Journal of King Saud University - Engineering Sciences*, 35(1), 53–61. <https://doi.org/10.1016/j.jksues.2021.02.015>
- Mütze, T. (2014). Scheduling with few changes. *European Journal of Operational Research*, 236(1), 37–50. <https://doi.org/10.1016/j.ejor.2013.11.011>
- Özpeynirci, S., Gökgür, B., & Hnich, B. (2016). Parallel machine scheduling with tool loading. *Applied Mathematical Modelling*, 40(9–10), 5660–5671. <https://doi.org/10.1016/j.apm.2016.01.006>
- Paiva, G. S., & Carvalho, M. A. M. (2017). Improved heuristic algorithms for the job sequencing and tool switching problem. *Computers & Operations Research*, 88, 208–219. <https://doi.org/10.1016/j.cor.2017.07.013>
- Pisinger, D., & Ropke, S. (2010). Large neighborhood search. In M. Gendreau & J.-Y. Potvin (Eds.), *Handbook of Metaheuristics* (pp. 399–419). Springer. https://doi.org/10.1007/978-1-4419-1665-5_13
- Privault, C., & Finke, G. (1995). Modelling a tool switching problem on a single NC-machine. *Journal of Intelligent Manufacturing*, 6, 87–94. <https://doi.org/10.1007/BF00123680>
- Privault, C., & Finke, G. (2000). k-server problems with bulk requests: An application to tool switching in manufacturing. *Annals of Operations Research*, 96(1–4), 255–269. <https://doi.org/10.1023/A:1018939132489>
- Raduly-Baka, C., & Nevalainen, O. S. (2015). The modular tool switching problem. *European Journal of Operational Research*, 242(1), 100–106. <https://doi.org/10.1016/j.ejor.2014.09.052>
- Rifai, A. P., Nguyen, H.-T., & Dawal, S. Z. M. (2016). Multi-objective adaptive large neighborhood search for distributed reentrant permutation flow shop scheduling. *Applied Soft Computing*, 40, 42–57. <https://doi.org/10.1016/j.asoc.2015.11.034>
- Rifai, A. P., Mara, S. T. W., & Norcahyo, R. (2022a). A two-stage heuristic for the sequence-dependent job sequencing and tool switching problem. *Computers & Industrial Engineering*, 163, Article 107813. <https://doi.org/10.1016/j.cie.2021.107813>
- Rifai, A. P., Sutoyo, E., Mara, S. T. W., & Dawal, S. Z. M. (2022b). Multiobjective sequence-dependent job sequencing and tool switching problem. *IEEE Systems Journal*, 17(1), 1395–1406. <https://doi.org/10.1109/JSYST.2022.3213767>
- Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472. <https://doi.org/10.1287/trsc.1050.0135>
- Schwerdfeger, S., & Boysen, N. (2017). Order picking along a crane-supplied pick face: The SKU switching problem. *European Journal of Operational Research*, 260(2), 534–545. <https://doi.org/10.1016/j.ejor.2016.12.037>

- Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In M. Maher & J.-F. Puget (Eds.), *Principles and Practice of Constraint Programming—CP98* (pp. 417–431). Springer. https://doi.org/10.1007/3-540-49481-2_30
- Shirazi, R., & Frizelle, G. D. M. (2001). Minimizing the number of tool switches on a flexible machine: An experimental study. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, 215(2), 253–261. <https://doi.org/10.1243/0954405011515190>
- Tang, C. S., & Denardo, E. V. (1988). Models arising from a flexible manufacturing machine, Part I: Minimization of the number of tool switches. *Operations Research*, 36(5), 767–777. <https://doi.org/10.1287/opre.36.5.767>
- Van Hop, N., & Nagarur, N. N. (2004). The scheduling problem of PCBs for multiple non-identical parallel machines. *European Journal of Operational Research*, 158(3), 577–594. [https://doi.org/10.1016/S0377-2217\(03\)00376-X](https://doi.org/10.1016/S0377-2217(03)00376-X)

Appendix

Appendix 1. Complete results of the experiment

Complete results of the experiment are attached in the supplementary material available at:
<https://polipapers.upv.es/index.php/IJPME/article/view/23073/18433>